



University of
Sheffield

VitalMonitor: IoT-driven Healthcare Management and Monitoring System

Vivek Vijay Choradia

Supervisor: Prof. Hamish Cunningham

*A report submitted in fulfilment of the requirements
for the degree of BSc in Artificial Intelligence and Computer Science*

in the

Department of Computer Science

May 25, 2024

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Vivek Vijay Choradia

Date: May 25, 2024

Abstract

This design and build project develops VitalMonitor, an Internet of Things as a Service (IoTaaS) platform. VitalMonitor incorporates non-intrusive wearable IoT technology, specifically, Smart Patch —into the attire of healthcare professionals working in high-stress environments such as operating theaters. The platform aims to facilitate continuous and real-time monitoring of vital signs to ensure the health and safety of the professionals, thereby directly enhancing patient care outcomes. The research addresses challenges faced in accuracy, reliability, and seamless integration of these technologies into existing healthcare workflows. Through a centralized health data management dashboard and using messaging API, the system supports immediate interventions and proactive health monitoring. This study expands the knowledge base on IoT applications in healthcare, emphasizing the critical impact of wearable technologies on the improvement of healthcare service delivery and professional health.

Acknowledgement

I am profoundly grateful to my supervisor, Prof. Hamish Cunningham, for his invaluable guidance and inspiration throughout this project. His expertise in the Internet of Things, as demonstrated in the COM3505 module, significantly enhanced my understanding and passion for IoT applications in healthcare. I also extend my thanks to Gareth Coleman, the visiting academic for the module, whose practical insights were immensely helpful.

My appreciation goes to Dr. Matt Elis, my second marker, whose constructive feedback helped refine my work. Special thanks to the Electronics Lab team — Cecil Wilkinson and Dr. Tom Howard — whose patience and expertise were crucial in overcoming the technical challenges I faced. Additionally, I acknowledge the Department of Computer Science for providing the necessary facilities and resources.

A special thank you to Siya Pradhan, a dear friend whose support were invaluable during this journey. I would like to thank my sisters Mallika and Malvika, and all my friends who believed in me and supported me throughout the dissertation journey.

And lastly but most importantly, I am also thankful to my parents for their endless encouragement and belief in my abilities.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aims and Objectives	1
1.3	Overview of the Report	2
2	Literature Review	4
2.1	Stress and Fatigue	4
2.2	Stress and Fatigue in Healthcare Settings	4
2.3	Current Strategies of Healthcare Institutions and their Inefficiencies	5
2.3.1	Integrated Care Systems (ICSs)	5
2.3.2	Understanding Evolving Needs	5
2.3.3	Workplace Interventions	6
2.3.4	Questionnaires	6
2.4	Measuring Stress and Fatigue Levels in Real-time	6
2.4.1	Current Practices	6
2.4.2	Next Best Alternative	7
2.5	Internet of Things (IoT)	7
2.6	IoT in Healthcare	7
2.6.1	Wearables	8
2.6.2	Limitations of Wearables	8
2.6.3	Non-Intrusive devices not used primarily in Healthcare	9
2.7	Internet of Things as a Service (IoTaaS)	9
2.8	Existing IoTaaS Platforms in Healthcare	10
2.8.1	Comparison between Existing Platforms	10
2.8.2	Limitations of Existing Platforms	10
3	Requirements and Analysis	12
3.1	Parameters for Healthcare Monitoring	12
3.2	VitalMonitor: System Description	13
3.2.1	Smart Patch	14
3.2.2	Dashboard	15
3.3	Requirements of the system	16

3.3.1	User Stories	17
3.3.2	Functional Requirements (Features)	18
3.3.3	Non-functional Requirements (Features)	19
3.4	Risk Analysis	20
3.5	Evaluation	22
4	Technology Stack	23
4.1	Firmware for Smart Patch	23
4.1.1	Computing Units - Microcontrollers (MCU)	23
4.1.2	Sensors	24
4.1.3	Connectivity Technology	27
4.2	Software for Smart Patch	28
4.2.1	Programming Environment	28
4.2.2	Programming Languages	29
4.3	Software for Web Dashboard	30
4.4	Messaging API: Twilio API	30
5	Design	31
5.1	System Design	31
5.1.1	Design Approach	31
5.1.2	Component Roles and Ideology	32
5.1.3	Data Flow Architecture	33
5.2	Database Design	34
5.2.1	Organizations	35
5.2.2	Devices	36
5.2.3	Users	36
5.2.4	DeviceData	36
5.3	Smart Patch (Firmware) Design	36
5.3.1	Hardware Components	36
5.3.2	Hardware Circuit Diagram	37
5.4	Middleware Design	39
5.4.1	Architecture and Responsibility	39
5.4.2	Design Principles	40
5.5	Dashboard Design	40
5.5.1	Technology and Tools	41
5.5.2	Design Pattern	41
5.5.3	User Flow	42
5.5.4	Wireframes	43
5.6	Network Communication and API Design	46
5.6.1	API and API Endpoints	46
5.6.2	Error Handling and Management	49

6	Implementation and Testing	50
6.1	Implementation and Testing Strategy	50
6.2	Firmware	50
6.2.1	Sensor Integration	53
6.2.2	MQTT and Adafruit IO Integration	55
6.2.3	Twilio Integration	55
6.3	Middleware	57
6.3.1	Connection with Google Cloud	57
6.3.2	Database Integration	58
6.3.3	API Integration	59
6.4	Web Dashboard	61
6.4.1	Session Management using custom decorators	61
6.4.2	Real-time Data	61
6.4.3	User Interface and Interaction	62
6.5	Testing	66
6.5.1	Acceptance Testing	66
6.5.2	System and Integration Testing	68
6.5.3	Unit Testing	69
6.5.4	Manual Testing	71
7	Results and Discussions	72
7.1	Revisited Requirements	72
7.2	Project Cost	73
7.3	VitalMonitor's IT Infrastructure	74
7.4	Prototype of Smart Patch	74
7.5	Limitations and Issues Encountered	75
7.5.1	Trade off between Sensor Accuracy and Non-Intrusiveness	75
7.5.2	Settings Get Lost When Power Goes Off	76
7.5.3	Converting Raw Data to Useful Data	76
7.5.4	Reliability depends on Network Bandwidth	77
7.5.5	Google Cloud	78
7.6	Further Work	79
7.6.1	Smart Patch: Final Product	80
7.6.2	Improving the Middleware Infrastructure	80
7.6.3	Algorithm to Measure Stress and Fatigue Levels	80
7.6.4	Interdisciplinary Collaboration for Enhanced Sensors	81
7.6.5	Prediction Model	81
8	Conclusion	83
	Bibliography	85

<i>CONTENTS</i>	vii
Appendices	89
A Software Development Lifecycle (SDLC)	90
B Legal Considerations	92
B.1 Data Protection Regulations	92
B.2 Medical Regulations	92
C Ethical Considerations	93
D Adafruit IO Dashboard	94
E Privacy Policy	95
F Terms and Conditions	96
G Manual Testing	97
H Generation of design idea using Dall-E	99
I Abbreviations	101

List of Figures

3.1	VitalMonitor: High-level working diagram	14
3.2	Device Placement	15
3.3	Key to Risk Rating	20
3.4	Risk Register	21
4.1	Comparison of Microcontrollers	24
4.2	LM-35 Sensor [31]	26
4.3	DS18B20 Sensor [32]	26
4.4	Connective Technologies compared against Bandwidth and Range [33]	28
5.1	Data Flow Diagram 1	33
5.2	Data Flow Diagram 2	34
5.3	Database Diagram	35
5.4	Circuit Diagram of Smart Patch	38
5.5	Legend for circuit diagram	38
5.6	Middleware	39
5.7	User Flow Diagram for Dashboard	42
5.8	Landing Page to register/login	43
5.9	Home Page	43
5.10	Add New Device page	44
5.11	Add New User Page	44
5.12	User Page	45
5.13	About Page	45
6.1	Comparison between V1 and V6 Firmware	51
6.2	Fetching User Health Data in real-time.	62
6.3	Login and Register Page (landing page)	63
6.4	Home Page	63
6.5	Add New Device Page	64
6.6	Add New User Page	64
6.7	User Page	65
6.8	About Page	65

6.9	CURL Command to test whether adding organization	69
6.10	Middleware Coverage	70
6.11	Web Dashboard Coverage	70
7.1	Overview of Requirements post development	73
7.2	Smart Patch	75
7.3	Data from sensors	78
7.4	Google Cloud Configurations	79
A.1	Anatomy of Agile Approach [37]	91
E.1	Privacy Policy	95
F.1	Terms and Conditions for VitalMonitor	96
H.1	Dall-E generated image	100

List of Tables

2.1	Comparison of IoTaaS Platforms in Healthcare	11
3.1	Comparison of Health Monitoring Parameters	13
3.2	Key to features	16
3.3	User Stories for VitalMonitor	17
3.4	Functional Features of the Smart Patch and Dashboard	18
3.5	Non-Functional Features of Smart Patch and Dashboard	19
4.1	Comparative study between Raspberry Pi Pico and ESP32-WROOM-32S3-Mini-1	24
4.2	Comparative Study between ECG Sensor and Pimoroni Pulse Sensor	25
4.3	Comparative Study between Connective Technologies	27
4.4	Comparative Study between Programming Environments	28
4.5	Comparative Study between Arduino C++ and MicroPython	29
4.6	Decision Matrix for Programming Languages	29
5.1	API Endpoints for Middleware	46
5.2	Dashboard API Endpoints	47
5.3	Smart Patch API Endpoints	48
5.4	Supported Error Codes	49
6.1	Summary of Firmware Iterations	52
6.2	User Stories for VitalMonitor Dashboard Implementation	66
6.3	Manual Test Cases for Smart Patch Device	71
7.1	Project Cost	74
G.1	Manual Test Cases for VitalMonitor Web Dashboard	97
G.2	Manual Test Cases for Smart Patch Device	98

Listings

6.1	Code for Pulse Sensor Integration	53
6.2	Code for DS18B20 Integration	53
6.3	Implementation of Moving Average to increase sensor accuracy	54
6.4	MQTT and Adafruit IO Client Configuration	55
6.5	Function to Send SMS via Twilio	56
6.6	Conditional SMS Notifications Based on Thresholds	56
6.7	Connection to Google Cloud MySQL using unix socket label	57
6.8	Secure Password Handling in Organization Model	58
6.9	API Endpoint for Organization Registration	59
6.10	API Endpoint for Adding or Updating a Device	59
6.11	API Endpoint for Real-Time Data Streaming	60
6.12	Custom Decorator - login_required	61
7.1	Using EEPROM in setup()	76

Chapter 1

Introduction

1.1 Motivation

Healthcare professionals (HCPs) are exposed to various sources of stress and pressure in their daily work, especially those who work in high-risk environments such as operation theatres. These stressors can negatively affect their physical, and mental health and well-being; as well as their performance and quality of care. Extensive research has been conducted on this topic indicating that HCPs are liable to experience a range of mental health issues, including depression [1], anxiety [2], burnout [3], and stress [4]. A study which surveyed 3,700 public sector workers in the United Kingdom found that staff working for the National Health Service were the most stressed, with 61% reporting feeling stress all or most of the time and as of 2015 59% stating that stress has increased from previous years [5].

The well-being concerns for HCPs elevated during COVID-19 and after it. Multiple researches were undertaken around the world talking about anxiety, stress, depression and the psychological well-being of HCPs [6][7]. This pressured healthcare institutions to work towards the health and well-being of their staff. More and more healthcare institutions worldwide are considering strategies for promoting staff safety and well-being. This project aims to create an Internet of Things as a Service (IoTaaS) Platform which helps these institutions get real-time insights into how to support HCPs in stressful environments best.

1.2 Aims and Objectives

The primary aim of this project is to design and implement VitalMonitor. VitalMonitor will be an integrated healthcare management system utilizing non-intrusive wearable technology for the real-time health monitoring of HCPs. This project targets professionals in high-stress environments, such as operating theatres and emergency rooms, where continuous health monitoring of the professionals is vital for maintaining high-quality patient care.

The objective would be to develop a wearable IoT device that measures vital without interfering with the HCPs' tasks, ensuring minimal disruption to their work. Additionally, the project will integrate this device within an Internet of Things as a Service (IoTaaS) platform, creating a seamless interface where health data is accessible in real-time through a web dashboard tailored for each healthcare institution.

With VitalMonitor, the project aspires to cultivate a proactive healthcare culture that consistently safeguards HCPs' welfare, establishing a foundation for improved patient care.

1.3 Overview of the Report

The introductory chapter establishes the foundation of this project, elucidating the motivation behind the study. It emphasizes the increasing awareness of healthcare professionals' health and well-being.

Chapter 2, provides literature review for the project, discussing the current strategies used by healthcare institutions and their inefficiencies. This chapter also discusses how IoT and IoTaaS came into the scenario and brought different wearable technologies which helped in digitalising healthcare but still has no solution that could help HCPs.

Chapter 3, navigates essential aspects of the project - essentially what will be developed. It provides an overview of how the system will work, what features will be implemented and success metrics. This chapter concludes with the risks identified.

Chapter 4, establishes the technical aspects of the project. Provides insights into what technologies will be needed for the success of the project. This chapter also includes a comparative study between various technologies for each component of the project and choosing the most suitable one for the project.

Chapter 5, gives a high-level overview of VitalMonitor and its components. It also discusses alternative design approaches to some of the requirements defined in Chapter 3. Chapters 3, 4 and 5 combined represents what could be seen as a brief Software and Hardware Requirements Specification (SHRS) document of the system, providing a foundational blueprint for its development and implementation.

Chapter 6, outlines the processes involved in the implementation and subsequent testing of the VitalMonitor system. It covers the development of firmware, middleware, and the user dashboard, employing an agile methodology to ensure adaptability and responsiveness throughout the project lifecycle. This chapter also describes various testing methodologies employed to validate the functionality and reliability of the system to ensure that all components meet the predefined specifications and work cohesively.

Chapter 7, discusses the final outcomes of the VitalMonitor system, highlighting the achievements and the impact of the solution. It critically analyzes the main limitations encountered during the project and suggests potential areas for future improvements. The evaluation of the system's performance and the feasibility of its application in real-world settings are also thoroughly examined.

Chapter 8, concludes the report by revisiting how developing this platform would help healthcare professionals and summarising why the proposed platform has shown potential to be widely used, if introduced to the market. The chapter reflects on the contributions of the project to the field of healthcare technology and outlines future research directions that could further enhance the system's functionality and marketability.

Chapter 2

Literature Review

This chapter critically explores the existing frameworks used by healthcare institutions to support the well-being of healthcare professionals (HCPs). It reviews the impact of stress and fatigue on HCPs within healthcare settings and evaluates how these challenges are currently being addressed. The chapter also highlights the significant role that emerging technologies, particularly the Internet of Things (IoT) and IoT-as-a-Service (IoTaaS), can play in enhancing the monitoring and management of healthcare environments. Additionally, it considers the shortcomings of current wearable technologies in providing non-intrusive, real-time health monitoring in remote settings. The review aims to underscore the necessity for innovative solutions that cater more effectively to the needs of HCPs. This would not only improve their well-being but the overall quality of patient care as well.

2.1 Stress and Fatigue

As per World Health Organization (WHO), Stress can be defined as a state of worry or mental tension caused by a difficult situation. Stress is a natural human response that prompts us to address challenges and threats in our lives. Everyone experiences stress to some degree. The way we respond to stress, however, makes a big difference to our overall well-being. Stress affects both the mind and the body. [8]

Fatigue is a term used to describe an overall feeling of tiredness or lack of energy. It isn't the same as simply feeling drowsy or sleepy. When you're fatigued, you have no motivation and no energy. Being sleepy may be a symptom of fatigue, but it's not the same thing. [9]

2.2 Stress and Fatigue in Healthcare Settings

Stress in healthcare professionals often stems from the high-pressure environment of medical settings, where the stakes are exceptionally high. Common stressors include a) long hours, b) complex patient interactions, and c) the emotional burden of continuous care giving. This

chronic stress can lead to psychological symptoms such as anxiety and depression, and physical symptoms like headaches and hypertension, ultimately impacting their work efficacy and personal wellbeing.

Fatigue among healthcare professionals is more than just feeling tired. It's a profound exhaustion that can result from prolonged physical, mental, and emotional labor. It is exacerbated by factors such as irregular shift patterns, the physical demands of the job, and the mental load of making critical decisions. Chronic fatigue can impair cognitive functions, reduce job performance, and increase the likelihood of errors in patient care.

In healthcare settings, stress and fatigue are exacerbated by the need to provide constant, high-quality care amidst frequently understaffed conditions. The relentless pace and emotional intensity of healthcare work can lead to burnout, a state of emotional, physical, and mental exhaustion. This not only affects the health of healthcare professionals but also compromises patient safety and the overall effectiveness of healthcare delivery. This made healthcare institutions to take various strategies to improve HCPs wellbeing.

2.3 Current Strategies of Healthcare Institutions and their Inefficiencies

The healthcare sector has been proactive in adopting various strategies to enhance the wellbeing of healthcare professionals (HCPs), acknowledging the multifaceted challenges they face. Despite the theoretical benefits of these strategies, practical inefficiencies and limitations often undermine their effectiveness. Sections listed below are different strategies which are used by healthcare institutions.

2.3.1 Integrated Care Systems (ICSs)

Integrated Care Systems (ICSs) have been a cornerstone in the UK's healthcare strategy. ICSs are designed to facilitate cohesive planning and service delivery across various healthcare domains, including both physical and mental health services, as well as social care. The overarching goal is to enable individuals to lead healthier and more independent lives through self-care and prevention [10]. However, ICSs sometimes struggle with delivering integrated care effectively, leading to fragmented services that are poorly coordinated around HCPs' needs. This fragmentation not only detracts from the patient and HCP experience but also results in inefficient resource use, echoing the financial and resource inefficiencies previously identified in Clinical Commissioning Groups (CCGs) [11], [12].

2.3.2 Understanding Evolving Needs

The **Understanding Evolving Needs** strategy, informed by Accenture's survey of 720 healthcare providers, seeks to align healthcare institutions with the shifting expectations of

HCPs, a need that became particularly acute during the COVID-19 pandemic [13]. While this adaptive approach is commendable, accurately identifying and addressing the evolving needs of HCPs presents significant challenges, with effectiveness varying based on numerous factors including the healthcare setting and the specific needs of HCPs [14], [15].

2.3.3 Workplace Interventions

Workplace Interventions, such as stress management programs, mindfulness practices, and workload adjustments, have been implemented to improve well-being and mitigate burnout among HCPs [16]. Nonetheless, their effectiveness can be inconsistent, largely dependent on how well these interventions are tailored and executed within the unique contexts of different healthcare settings [17].

2.3.4 Questionnaires

The use of **Questionnaires** to assess the health and well-being of HCPs reflects a direct approach to understanding their needs. However, this method often fails to capture real-time changes and may elicit responses that do not accurately reflect the HCPs' health status [18]. HCPs might prioritize patient care over their well-being or misjudge their health status as balanced, leading to responses that may not be entirely accurate or reflective of their actual needs.

The inefficiencies and limitations inherent in these strategies underscore the pressing need for innovative solutions that offer more immediate and comprehensive support for HCPs. One promising direction is the adoption of **Wearable Technologies**, an Internet of Things (IoT) application in healthcare, which could provide real-time health monitoring and personalized support, addressing many of the current methodologies' shortcomings.

2.4 Measuring Stress and Fatigue Levels in Real-time

Currently, there is no universally accepted algorithm for precisely calculating stress and fatigue levels, as these conditions are influenced by a complex interplay of physical, psychological, and environmental factors. This lack of a standardized measurement approach makes it challenging to accurately assess and manage stress and fatigue, particularly in high-pressure environments like healthcare settings.

2.4.1 Current Practices

Technological advancements have led to the development of devices that can measure indicators of stress in real-time. One notable example is the use of Electrodermal Activity (EDA) testing, popularized by devices like Fitbit. The EDA test measures the skin's electrical activity, which varies with the body's sweat level—a physiological response associated with stress. Fitbit's implementation of this technology assesses stress levels by requiring users

to engage in a two-minute meditation session, during which the device monitors changes in EDA. [9] However, this method poses practical challenges for healthcare professionals (HCPs) who often cannot spare two uninterrupted minutes for such a test while on duty. Therefore, while valuable, this technology may not always be suitable for real-time stress monitoring in fast-paced healthcare environments.

2.4.2 Next Best Alternative

To adapt real-time stress and fatigue monitoring for healthcare settings, technologies need to utilize parameters that can be passively monitored without disrupting workflow. These parameters should be easily sensed via remote and non-intrusive devices, allowing HCPs to wear and use them continuously throughout their duties. By “non-intrusive,” it means the device should be lightweight, comfortable, and not interfere with their work.

Sensors which can detect such parameters should be integrated into devices that are easy to use and interpret. This ensures that HCPs can benefit from the insights provided without needing extensive training or causing disruption to their critical work. And one of the leading technologies which can help making such a device is Internet of Things.

2.5 Internet of Things (IoT)

The Internet of Things (IoT) describes the network of physical objects — “things” — that are embedded with sensors, software, and other technologies to connect and exchange data with other devices and systems over the Internet. These devices range from ordinary household objects to sophisticated industrial tools. With more than 7 billion connected IoT devices today, experts are expecting this number to grow to 22 billion by 2025. [18]

IoT devices generate vast amounts of data - offering valuable insights for businesses and individuals. With advances in machine learning and analytics, coupled with IoT data, businesses can gather insights faster and more easily. The emergence of these allied technologies continues to push the boundaries of IoT and the data produced by IoT also feeds these technologies. [18]

2.6 IoT in Healthcare

The Internet of Things (IoT) in healthcare, also known as the Internet of Medical Things (IoMT), refers to it's a network of medical devices, software, and tech solutions that can monitor patient health, manage treatments, and even assist in surgical procedures [19]. These devices are equipped with internet connectivity, enabling them to communicate with healthcare systems and each other. This integration facilitates a broad spectrum of health services and medical applications.

IoMT encompasses a variety of devices from wearable health monitors to advanced surgical equipment. These tools collect, analyze, and transmit health data, providing real-time insights crucial for informed decision-making and tailored patient care. The use of IoMT leads to greater accuracy in diagnoses, improves treatment outcomes, and enhances the efficiency of healthcare operations.

2.6.1 Wearables

Wearables are IoT devices that can be worn or carried on the body. These devices can be separated into categories such as smartwatches, wristbands, and hearables. The personal data of the user can be monitored and measured through powerful microchips and smart sensors that are embedded within the wearable device. Connecting to additional devices using Bluetooth, WiFi or a cellular network can further enhance the user experience [20]. They are the most used device for healthcare monitoring.

- **Smart Watch:** Smart watches stand out in the wearable technology landscape for their versatility and widespread adoption. As per Statista, 2023, smartwatches currently dominate the wearables market and have held a share of around 45 per cent since 2018. Most fitness trackers are built into smartwatches and can track steps, distance, heart rate, calories burnt, and other fitness metrics. They offer a comprehensive suite of features that go beyond fitness tracking to include notifications, apps, and sometimes even phone capabilities. Their appeal lies in their ability to provide a snapshot of an individual's health and daily activities directly from their wrist.
- **Chest Strap:** Chest straps are specialized wearables designed for precise heart rate monitoring. Unlike wrist-based devices, chest straps measure electrical signals directly from the chest, offering a higher level of accuracy. They are particularly favored by athletes and individuals engaged in rigorous physical activities for monitoring heart rate and performance metrics.

2.6.2 Limitations of Wearables

1. Remoteness

One significant limitation of current wearable technology is its remoteness or isolation from centralized healthcare systems. This disconnect hampers the potential for wearables to inform clinical decisions and provide a holistic view of a patient's health.

While wearable devices are capable of collecting a broad spectrum of health data from healthcare professionals themselves, there remains a significant barrier in integrating this data with the centralized systems used in hospitals and clinics. This is because most of such devices are made with general consumers in mind and the manufacturers don't usually sell it to organization with the perspective of monitoring health of all the staff in an organization. This disconnect hinders the ability of these devices to inform

the organization about the health and fatigue levels of healthcare professionals in real time, which could otherwise support better staffing decisions and reduce the risk of burnout and medical errors.

2. Non-Intrusiveness

In healthcare settings, the non-intrusiveness of wearables is crucial. However, even state-of-the-art smartwatches face restrictions in sensitive environments like operation theatres or Intensive Care Units (ICUs). According to NHS England’s guidance on uniforms and workwear, wearing watches in clinical settings is discouraged due to the risk of harboring microorganisms and hindering effective hand hygiene [21]. Moreover, wristwatches may interfere with medical procedures, posing risks during surgeries or delicate operations.

2.6.3 Non-Intrusive devices not used primarily in Healthcare

Healthcare devices are subject to rigorous regulatory scrutiny to ensure they meet high standards of accuracy, reliability, and safety. The certification process for medical devices, which includes clinical trials and detailed documentation, is both time-consuming and expensive. This rigorous scrutiny is essential to prevent errors that could directly affect patient health but creates a significant barrier to entry for new and innovative non-intrusive technologies. Furthermore, different countries and regions have their own regulatory frameworks, which can complicate the process for manufacturers looking to market their devices globally.

While non-intrusive devices offer the convenience of passive monitoring, maintaining the accuracy and reliability of these devices remains a technological hurdle. Sensor technology, particularly in non-intrusive formats, must evolve to capture data with the same precision as more traditional, intrusive methods. For example, sensors that work under or through clothing or that can monitor biometrics without direct skin contact often face issues such as signal attenuation or interference. These limitations can result in less reliable data, which in turn may affect the clinical utility of the information these devices provide.

Therefore, no such device exists which is remote and non-intrusive which can be used for monitoring vitals of healthcare professionals while they are working.

2.7 Internet of Things as a Service (IoTaaS)

Internet of Things-as-a-Service is one of the latest iterations in the “as-a-service” model that has become one of the popular technologies [22]. IoTaaS, or IaaS, or Internet of Things as a Service, integrates IoT technology into a service model, allowing companies to use IoT solutions without managing the infrastructure. It became notably popular in the mid-2010s as businesses sought to enhance efficiency and data collection without the overhead of developing proprietary IoT systems.

IoTaaS has been a subject of extensive research and development, as evidenced by its frequent spotlight at the EAI International Conference on IoT as a Service and Consumer Electronics Showcase. The EAI IoTaaS 2024 conference, for instance, is set to highlight cutting-edge advancements in IoT technologies and their applications across various industries. These conferences serve as pivotal forums for exchanging ideas and fostering international collaboration among scholars and industry experts, thereby propelling the global IoT industry forward.[23]

2.8 Existing IoTaaS Platforms in Healthcare

IoTaaS has become a transformative force in healthcare, enabling sophisticated remote patient monitoring systems and real-time data transmission to medical professionals. Such capabilities have improved the management of chronic diseases, improving patient outcomes, and enhancing operational efficiencies within healthcare facilities.

Notable research presented at the EAI International Conference on IoTaaS 2017, such as the “Home Healthcare Matching Service System Using the Internet of Things,” illustrates the potential of IoTaaS to deliver medical care in more comfortable, patient-preferred settings outside of traditional hospital environments.[24] This shift not only aligns with broader trends towards patient-centered healthcare but also highlights the adaptability and potential of IoTaaS to revolutionize medical care practices.

A lot of HealthTech Companies are making their mark by creating IoTaaS platforms for healthcare. Some of the major platforms are Philips HealthSuite, GE Healthcare’s Edison Platform, and Cisco Healthcare IoT. These platforms integrate with state-of-the-art healthcare technologies to get patient’s data and uses their home AI and data analytics skills to provide the data to the doctors.

2.8.1 Comparison between Existing Platforms

A comparison table is shown between some of the current industry-standard platforms in provided in Table 2.1. Some of these platforms call them Platform-as-a-Service because they only provide the virtual platform using pay-per-use model to the end organizations. In the table N/A means no information was available on the sources about that feature for that platform.

2.8.2 Limitations of Existing Platforms

As shown in the Table 2.1, all the platforms do not have a focus on HCP well-being. While these platforms employ advanced technologies to enhance healthcare management, they uniformly lack features specifically tailored to monitor the health and well-being of healthcare

Features	Philips Health-Suite	GE Health-care's Edison Platform	Cisco Health-care IoT
Secure Data Management	✓	✓	✓
Provides advanced analytics and AI capabilities	✓	✓	✓
Scalable	✓	✓	N/A
Cloud-based	✓	✓	✗
Multiple Devices can be Integration	✓	✓	N/A
Security	N/A	N/A	✓
Real-time data & Contextualized Insights	✓	✓	✓
Focus on HCP Well-being use case	✗	✗	✗

Table 2.1: Comparison of IoTaaS Platforms in Healthcare

providers themselves. This oversight indicates a market opportunity for the development of a platform that addresses the needs of healthcare professionals, ensuring their well-being is also supported and monitored effectively.

Chapter 3

Requirements and Analysis

In response to the pressing need for seamless healthcare monitoring, this project introduces VitalMonitor, a centralized system designed to enhance real-time monitoring of healthcare professional (HCP) vitals. This innovative IoTaaS platform comprises two key components: the Smart Patch, a wearable IoT device for vital sensing, and a comprehensive dashboard for monitoring and managing healthcare professionals. VitalMonitor not only simplifies access to vital health metrics but also integrates these functionalities into a user-friendly platform, minimizing the technological burden on healthcare organizations and easily integrate into their existing workflow. The chapter covers on the required features of the system, risk analysis and an evaluation system for VitalMonitor is introduced.

3.1 Parameters for Healthcare Monitoring

To effectively monitor stress and fatigue levels in real-time, the Smart Patch will utilize two key physiological parameters. The selection of these parameters is crucial to ensure accurate and actionable health insights for healthcare professionals (HCPs) working in demanding environments. Identifying relevant parameters is vital to the Smart Patch's efficacy. The VitalMonitor project focuses specifically on stress and fatigue levels because of their significant impact on healthcare professionals. A range of physiological and psychological indicators were considered, but only those with high relevance and monitoring feasibility were selected.

A comparative analysis of potential monitoring parameters was conducted based on relevance to stress and fatigue and the feasibility of non-intrusive monitoring. Table 3.1 presents the results of this analysis. Based on this analysis, Heart Rate Variability (HRV) and Body Temperature are identified as the most relevant parameters for monitoring stress and fatigue levels. These parameters offer the advantage of continuous, non-intrusive monitoring through the Smart Patch.

Parameter	Relevance to Stress/Fatigue	Monitoring Feasibility
Heart Rate Variability (HRV)	High	High
Body Temperature	Moderate	High
Blood Pressure	High	Moderate
Galvanic Skin Response	High	Moderate
Cortisol Levels	High	Low

Table 3.1: Comparison of Health Monitoring Parameters

While HRV provides detailed insights into an individual’s stress and fatigue levels, it is challenging to calculate due to the following reasons:

- **Sensor Accuracy:** HRV requires highly accurate sensors to capture subtle variations in heart rhythm.
- **Noise Interference:** Movement and other physiological signals can interfere with the accuracy of HRV measurements.
- **Data Processing:** Advanced algorithms are required to accurately interpret HRV data.

Therefore, for simplicity and ease of implementation, Heart Rate (beats per minute) will be used instead of HRV. Although this approach provides slightly less nuanced information, it still allows effective monitoring of stress and fatigue levels.

3.2 VitalMonitor: System Description

VitalMonitor marks a significant advancement in healthcare technology by prioritizing the well-being of HCPs in high-stress environments. This system includes the Smart Patch, a discrete, non-intrusive wearable device tailored for healthcare settings, and a web-based dashboard that facilitates real-time data analysis and monitoring. The goal is to deliver actionable insights into HCP health, enabling timely interventions to alleviate stress and fatigue, and thereby enhancing patient care and safety.

VitalMonitor will provide passive health monitoring of HCPs to the organization admins. Passive monitoring refers to the process of collecting data from individuals without requiring their active participation or input. In healthcare, passive monitoring typically involves the use of sensors or other technology to continuously gather data on various physiological parameters. This doesn’t require the need for individuals to perform any specific actions or interactions.

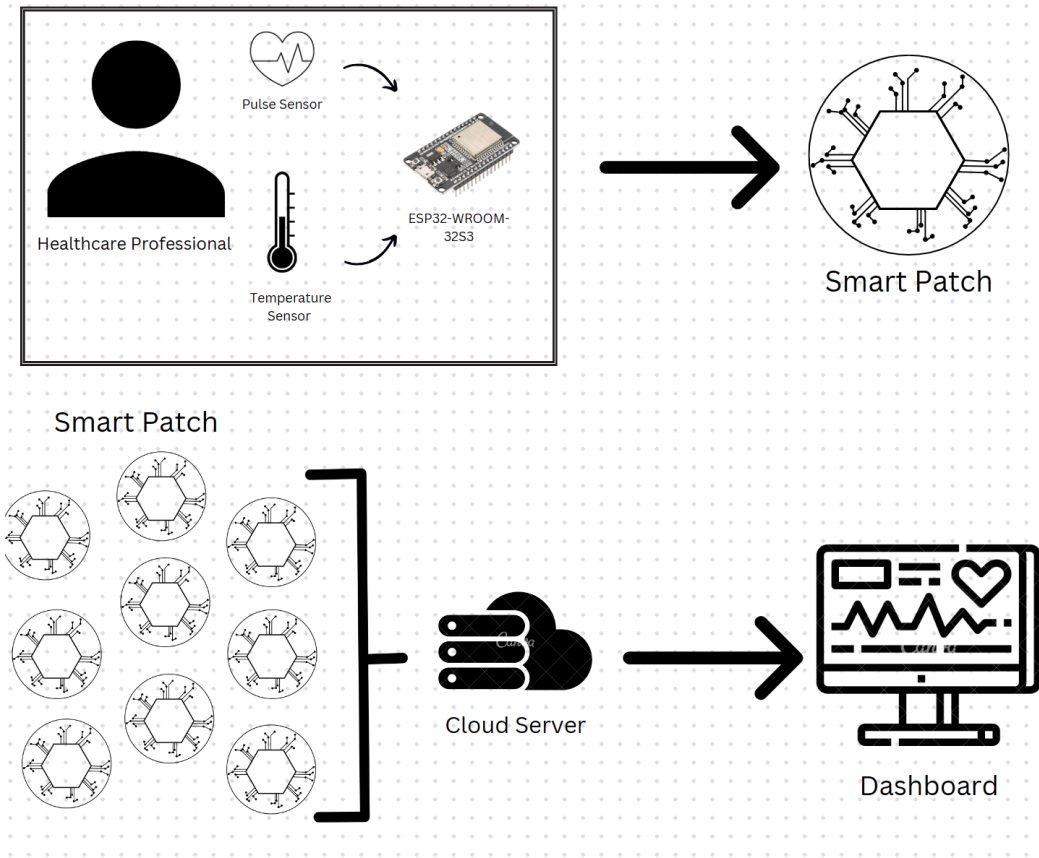


Figure 3.1: VitalMonitor: High-level working diagram

Figure 3.1 visually represents how VitalMonitor will work all in all and how different components of VitalMonitor will interact and work together.

3.2.1 Smart Patch

The Smart Patch is a non-intrusive wearable IoT device which will be designed to monitor the health and well-being of healthcare professionals in high-stress environments. This innovative IoT device aims to address the limitations of existing wearable technologies, particularly the restrictions on wearing smartwatches in healthcare settings.

While stress and fatigue are complex and multifaceted, heart rate and body temperature serve as surrogate markers that can be effectively measured in real-time. The integration of these parameters into the Smart Patch enables continuous monitoring, offering valuable insights into HCPs' health and well-being. By focusing on these indicators, the Smart Patch aims to provide actionable data that empowers healthcare institutions to proactively address stress and fatigue, ultimately enhancing the overall quality of care in high-stress environments.

Device Placement

The placement of the Smart Patch is crucial to ensure reliable vital sign measurements while minimizing interference with healthcare professionals' tasks. Research in clinical physiology suggests that the ear lobe is an ideal site for accurate and consistent heart rate measurement due to its rich vascular supply and proximity to the surface, which makes it less susceptible to external noise interference and movement artifacts. Additionally, the area behind the ear is well-suited for measuring body temperature, as this region is less exposed to ambient temperature variations than other body surfaces, providing a more accurate reading of core temperature.

Moreover, the ear region provides an advantageous, non-intrusive position where the device is unlikely to interfere with daily tasks. This ensures the comfort and continuous monitoring of HCPs. Figure 3.2 illustrates the optimal placement of the Smart Patch, highlighting the exact positions on the ear lobe (for heart rate measurement) and behind the ear (for temperature sensing) in red.

By positioning the Smart Patch in these locations, the system can reliably collect data while ensuring that healthcare professionals can work without distraction or discomfort.



Figure 3.2: Device Placement

3.2.2 Dashboard

To facilitate real-time monitoring, a web dashboard will be made where the institutions can track the vitals of all the HCPs working at that time using the data being collected by smart patch in real-time. It will present options to get alerts when an HCP's vital signs go beyond the threshold. Overall, the dashboard serves as a central hub for visualizing and interpreting the data collected by the smart patch, facilitating informed decision-making and proactive measures to support the health and well-being of HCPs in high-stress environments.

Agile methodology of the Software Development Lifecycle (SDLC) (refer to **Appendix A**) will be used for creating both the Smart Patch and the dashboard.

3.3 Requirements of the system

Both the Smart Patch and the Dashboard boast a set of functional and non-functional requirements meticulously designed to ensure seamless real-time monitoring by healthcare professionals (HCPs). These are categorized as follows:

- **User Stories**

User stories capture the requirements from the user’s perspective, providing insights into their needs. The stories are grouped by user roles. These user stories, along with their priorities, are presented in Table 6.2.

- **Functional Features**

Functional features represent the capabilities and behaviours of a system that directly contribute to its primary purpose and functionality. These features are concerned with what the system does and how well it performs its intended tasks. These features are listed in Table 3.4.

- **Non-Functional Features**

Non-functional features, on the other hand, are characteristics of a system that do not directly relate to its specific functionalities but rather focus on how well the system performs its functions. These features are listed in Table 3.5.

Additionally, they are prioritized using the MoSCoW prioritization framework into Must-Have, Should-Have, Could-Have, and Won’t-Have categories. Key to priority framework is given in Table 3.2.

Category	Description
Must-Have	This category consists of features that are “must” for the system. They represent non-negotiable needs for the system.
Should-Have	They are essential for the system, but they are not vital. If left out, the system still functions. However, the features may add significant value.
Could-Have	These features are not necessary to the core function of the system.
Won’t-Have	This category contains features which will not be included in a specific release, time frame or prioritised in the future. Their absence is not detrimental to the functionality of the final product.

Table 3.2: Key to features

3.3.1 User Stories

User Role	Story	Priority
Admin	I want to register my organization.	Must Have
Admin	I want to log in to the dashboard.	Must Have
Admin	I want to add new users.	Must Have
Admin	I want to add new devices.	Must Have
Admin	I want to manage user-device assignments.	Should Have
Admin	I want to monitor vital signs in real-time.	Must Have
Admin	I want to securely log out.	Should Have
Admin	I want to view a summary of users and devices.	Could Have
User	I want to wear the device comfortably on my ear.	Won't Have
User	I want to easily activate the Smart Patch by pressing a button.	Should Have
User	I want to press the button again to stop sending data.	Should Have
User	I want the device to continuously measure my heart rate and body temperature while activated.	Must Have

Table 3.3: User Stories for VitalMonitor

3.3.2 Functional Requirements (Features)

Feature	Description	MoSCoW Priority
Smart Patch Features		
Real-time Monitoring	Continuously monitors vital signs (HRV and body temperature) in real-time, providing instant feedback on the wearer's physiological status.	Must-Have
Heart Beat Rate Monitoring	Monitors heart beat rate of the wearer in real-time.	Must-Have
Body Temperature Monitoring	Monitors body temperature of the wearer in real-time.	Must-Have
Wireless Connectivity	Equipped with wireless capabilities to securely transmit data to a cloud platform, enabling remote access for healthcare institutions.	Must-Have
Dashboard Features		
Real-time Monitoring	Displays real-time heart rate and body temperature data of HCPs, allowing institutions to track vital signs continuously.	Must-Have
Alerts and Threshold	Enables institutions to set and receive alerts when an HCP's vital signs surpass predefined thresholds, facilitating timely intervention in case of abnormal patterns.	Must-Have
Multiple HCP Tracking	Supports simultaneous monitoring of multiple HCPs, providing a comprehensive overview of the health status of all professionals working at a given time.	Should-Have
User Authentication	Implements secure user authentication to ensure authorized access, maintaining the confidentiality and integrity of healthcare data.	Could-Have
Customizable User Permissions	Provides customizable user permissions, allowing institutions to control access levels for different roles, ensuring data privacy and security compliance.	Could-Have

Table 3.4: Functional Features of the Smart Patch and Dashboard

3.3.3 Non-functional Requirements (Features)

Feature	Description	MoSCoW Priority
Smart Patch Non-Functional Features		
Non-Intrusiveness	Designed to be comfortable for prolonged wear, lightweight, unobtrusive, and adhering to safety and hygiene standards. Placed on the body where least intrusive for healthcare professionals.	Must-Have
Device Portability	Designed to be easily portable, allowing healthcare professionals to move freely and perform their duties without hindrance.	Must-Have
Regulatory Compliance	Adheres to relevant healthcare and IoT regulatory standards to ensure the device's safety, effectiveness, and compliance with legal requirements.	Must-Have
Long Battery Life	Designed with a long-lasting battery to minimize the need for frequent recharging, ensuring continuous monitoring during extended shifts.	Should-Have
Durability	Constructed with durable materials to withstand daily wear and tear in a healthcare environment, ensuring the device's longevity and reliability.	Should-Have
Compatibility	Ensures compatibility with various body types and sizes, accommodating diverse healthcare professionals with different preferences and needs.	Should-Have
Dashboard Non-Functional Features		
User-Friendly Interface	Designs an intuitive and user-friendly dashboard interface, enhancing the overall experience for healthcare professionals and administrators interacting with the monitoring system.	Must-Have
Performance Optimization	Optimizes the dashboard's performance to handle real-time data efficiently, ensuring a smooth and responsive user experience even during periods of high data traffic.	Should-Have
Scalability	Designs the dashboard to be scalable, accommodating potential future expansions in the number of monitored healthcare professionals without compromising performance or user experience.	Should-Have
Security and Compliance	Ensures data security by implementing robust measures to protect sensitive healthcare information, complying with relevant regulations and industry standards.	Should-Have

Table 3.5: Non-Functional Features of Smart Patch and Dashboard

3.4 Risk Analysis

This section lists down all the potential risks that this project might face. The risk register provided in Figure 3.4 is continuously updated throughout the project timeline as per the key to risk rating provided in Figure in 3.3.

Key to Risk Rating:

Likelihood (L)	Description
5	Very likely/imminent - certain to happen.
4	Probable - a strong possibility of it happening
3	Possible - it may have happened before
2	Unlikely - could happen but unusual
1	Rare - highly unlikely to occur

Impact (I)	Description
5	Catastrophic:non-functional system
4	Major - significant changes to the system
3	Moderate - moderate changes to the system
2	Minor - minor changes to the system
1	Very minor - very minor changes to the system or requirements.

		Impact (I)				
Likelihood (L)		1	2	3	4	5
	5	5	10	15	20	25
	4	4	8	12	16	20
	3	3	6	9	12	15
	2	2	4	6	8	10
	1	1	2	3	4	5

Risk Rating (RR)	Action
High Risk	Stop the task/activity until controls can be put into place to reduce the risk to an acceptable level.
Medium Risk	Determine if further precautions are required to reduce risk to as low as is reasonably practicable.
Low Risk	No further action, keep under review.

Figure 3.3: Key to Risk Rating

Risks	Consequences	Risk Rating			Control Measures	Residual Risk		
		L	I	RR		L	I	RR
Inadequate defines requirements	Can lead to missing core features and can result in a unsuccessful project	4	5	20	Ensure that the requirements are well-defined. Following an iterative development will allow to incorporate last minute changes	2	1	2
Lack of on-time feedback from the supervisor	Can potentially impact the quality of the project	3	4	12	Submitting drafts at least a week before the deadline. Setting up regular meetings This would give both parties enough time time to suggest/make changes	1	4	1
Extenuating circumstances	Can potentially lead to a delay	3	3	9	Communicate with you supervisor and take appropriate steps (extending the deadline)	1	3	3
Delay in shipment of sensors	Can potentially lead to a delay	4	4	16	Ordering sensors before actual implementation phase. Can contact another supplier for immediate delivery.	2	2	4
Issues with IDE	Can lead to a delay in the development stage	4	5	20	Checking version compatibilities. Using the most up-to-date versions - this would make sure that bug fixes are applied immediately.	3	2	6
Compatibility Issues between ESP32 and other sensors	This may take extra time to solve the issues or find another component to replace the faulty ones.	3	5	10	Compatibility study will be done before starting actual implementation. Will ask the supervisor to help me with it, refer to documentations to debug the issue. If not, will look out for other compatible components and order it ASAP.	3	2	6

Figure 3.4: Risk Register

3.5 Evaluation

To ensure the robustness, reliability, and effectiveness of the Smart Patch and the dashboard, a comprehensive testing and quality assurance process will be implemented throughout the implementation and testing stage of the project:

- **Unit tests:** Conducted to validate the correctness of individual components and the functionality of the Smart Patch and dashboard.
- **Integration testing:** Conducted to verify the seamless integration of components and modules within the Smart Patch and the dashboard.
- **System testing:** Conducted to assess the overall functionality and the performance of the integrated Smart Patch and the dashboard.
- **Acceptance Testing:** Covering all user stories to ensure that the system meets user requirements.
- **Manual testing:** For features that cannot be adequately tested using the above tests.

A systematic evaluation of legal and ethical considerations will also take place:

1. **Legal considerations:** Medical Regulations outlined by MHRA and GDPR guidelines will be followed (see **Appendix B**).
2. **Ethical considerations:** Guidelines provided by the University of Sheffield will be followed, and an ethics application will be made if the project starts to use real human health data other than the developers' data (see **Appendix C**).

Chapter 4

Technology Stack

This chapter will cover on technology stack needed to build the smart patch and the dashboard. It will provide an overview of what might be needed, possible technologies to choose from and choosing the most suitable ones for the development of the system.

4.1 Firmware for Smart Patch

4.1.1 Computing Units - Microcontrollers (MCU)

A microcontroller (MCU) is a compact, self-contained computer integrated on a single chip, designed for embedded applications with lightweight processing needs [25]. MCUs are known for their small size, low power consumption, and cost-effectiveness. MCUs require a hands-on approach, involving configuration for both hardware (attachment of components, sensors, wires, etc.) and software (burning firmware, incorporating libraries, installing drivers, etc.). MCUs lack a conventional operating system and utilize firmware, a special code burned onto the device to execute specific functions and ensure proper hardware functionality. While they cannot serve as personal computers, some advanced MCUs offer features like networking, making them appealing for projects with specific requirements.

There are multiple MCUs available in the market but the most popular ones are Raspberry Pi Pico and ESP32-WROOM-32S3-Mini-1. A comparative study between the two is provided in Table 4.1 to help in choosing the ideal MCU.

Comparing the two, ESP32-WROOM-32S3-Mini-1 comes out as ideal for the Smart Patch because of its in-built wireless connectivity. It is important to have such connectivity to provide portability of the device so that wearer can wear it all day and still can go around the campus.

To power the microcontroller, either direct power is provided through the C-pin or a LiPo battery shall be utilized for portability and extended use.



(a) Raspberry Pi Pico [26]



(b) ESP32 [27]

Figure 4.1: Comparison of Microcontrollers

Feature	Raspberry Pi Pico	ESP32-WROOM-32S3-Mini-1
Manufacture	Raspberry Pi Foundation	Espressif Systems
Architecture	ARM Cortex-M0+	Xtensa LX6
Processor Clock Speed	Up to 133 MHz	Up to 240 MHz
Wireless Connectivity	None	Wi-Fi, Bluetooth (BLE)
Memory	2MB Flash (No built-in RAM)	2MB Flash
GPIO Pins	26	Approximately 38
Development Environment	MicroPython, C/C++ with Pico SDK	Arduino IDE, PlatformIO, Espressif IDF

Table 4.1: Comparative study between Raspberry Pi Pico and ESP32-WROOM-32S3-Mini-1

4.1.2 Sensors

MCUs typically come equipped with general-purpose I/O pins (GPIO), allowing users to connect various external peripherals to the computer. Examples of common input devices include sensor modules, microphones, and cameras, while output devices encompass LEDs, speakers, displays, and more. Users can program these individual pins to receive data.

Heart Rate Sensors

To calculate HRV, heart rate sensors have to be used and connected with the MCU. Two sensors which are used abundantly in the industry, especially healthcare are ECG Sensors and Pimoroni Pulse Sensor. A comparative study is provided in Table 4.2 to help in determining the ideal sensor.

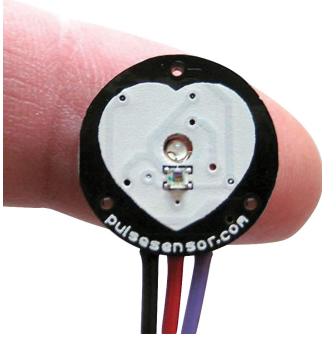
Features	ECG Sensor	Pimoroni Pulse Sensor
Working Principle	ECG Sensor measures the heart's electrical activity over time [28]. It detects the heartbeat rhythm regularly to keep a check on the health of your heart [29].	Pimoroni Pulse Sensor uses a technique called photoplethysmography (PPG) to detect the change in the colour of your skin with each beat of your heart when the sensor is pressed against your fingertip [30].
Use Cases	It can detect Arrhythmias like Atrial fibrillation and can also warn you about heart attack using strokes and movements around the heart.	It can be used by students, artists, athletes, makers, and game & mobile developers who want to easily incorporate live heart-rate data into their projects.
Compatibility	Works with most smartwatches.	Works with Arduino, micro:bit, littleBits, and most maker platforms.
Intrusivity	Non-invasive requires skin contact.	Less intrusive, fingertip contact.
Additional Features	Some devices can export an ECG graph of your heart rate, which can be a huge help when talking to your doctor.	The Pulse Sensor has 3 holes around the outside edge which make it easy to sew it into almost anything.
Image	 A photograph of an ECG sensor kit. It includes a small red circuit board with a black cable attached. The cable has three colored loops (red, green, yellow) at the end. There are also several white circular electrodes with green circuitry.	 A photograph of a Pimoroni Pulse Sensor. It is a small, circular, black device with a white heart-shaped sensor area in the center. It is being held against a person's fingertip. The device has three colored wires (red, green, blue) extending from the bottom. The text 'pulsesensor.com' is visible on the black casing.

Table 4.2: Comparative Study between ECG Sensor and Pimoroni Pulse Sensor

One of the main features of the Smart Patch is non-intrusivity. While both the sensors are efficient and used widely, Pulse Sensor comes out as a perfect choice for Smart Patch because not only it is less intrusive but also it works well with ESP32 (Arduino).

Body Temperature Sensor

To calculate body temperature, thermometers are used normally in healthcare. But thermometers are not digital devices and therefore can't be connected to the smart patch. Other sensors which are widely used for sensing temperature are thermistors, LM-35 sensors and DS18B20. Thermistors are more temperature-sensitive resistors and not sensors and require a readout circuit, which makes it harder to integrate them seamlessly into Smart Patch.

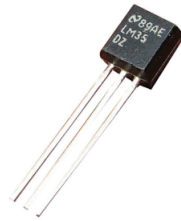


Figure 4.2: LM-35 Sensor [31]

On the other hand, the LM-35 Sensor is a precision IC temperature sensor. It provides a linear voltage vs temperature characteristic making it easier to figure out the temperature. It can operate over a temperature range of 55°C to 150°C and has an accuracy of ± 1 degree Celsius. Additionally, its analog output allows for simple integration with the analog-to-digital converter of the microcontroller used in the Smart Patch.



Figure 4.3: DS18B20 Sensor [32]

Lastly, the DS18B20 sensor is a digital temperature sensor with a unique 1-wire interface that allows it to communicate efficiently with the Smart Patch's processing unit. It operates

in the range of $-55^{\circ}C$ to $125^{\circ}C$ and boasts an accuracy of $\pm 0.5^{\circ}C$ over the range of $-10^{\circ}C$ to $85^{\circ}C$. Additionally, the DS18B20 supports multiple devices on the same bus, enabling the monitoring of multiple sensors using only one microcontroller input. Its digital output makes it immune to noise and interference, providing reliable measurements.

Both the LM-35 and DS18B20 have notable features. The project will test both sensors and retain the one that demonstrates superior accuracy and reliability.

4.1.3 Connectivity Technology

Connective Technology is nothing but a network of connected “smart” devices that communicate seamlessly over the internet. Wi-Fi, Bluetooth and LoRaWAN are the most used networks associated with IoT. Table 4.3 provides a comparative study between the three to help in choosing the most suitable technology for the project.

Features	Wi-Fi	Bluetooth	LoRaWAN
Scalability	Limited for massive IoT.	Mesh supports large networks.	Scales well for wide area.
Power Efficiency	High power consumption.	Low Energy, efficient.	Low power usage.
Range and Coverage	Short to medium range.	Moderate, indoor range.	Long-range, wide coverage.
Data Rate	High-speed data transfer	Moderate data rates	Low data rates
Cost and Infrastructure	Existing infrastructure may be costly.	Cost-effective, integrated widely.	Low infrastructure cost
Security	Robust encryption, authentication	Secure with enhancements	Implement AES encryption

Table 4.3: Comparative Study between Connective Technologies

From the comparison in Table 4.3 and Figure 4.4, LoRaWAN stands out as a clear and most efficient choice for Smart Patch. But the only disadvantage is that this network is not usually found in healthcare institutions compared to Wi-Fi which is available in nearly every healthcare institution. This advantage of Wi-Fi makes it easier to integrate Smart Patch with existing resources of institution thereby saving extra cost and other resources. Hence, Smart Patch will use Wi-Fi as its connective technology.

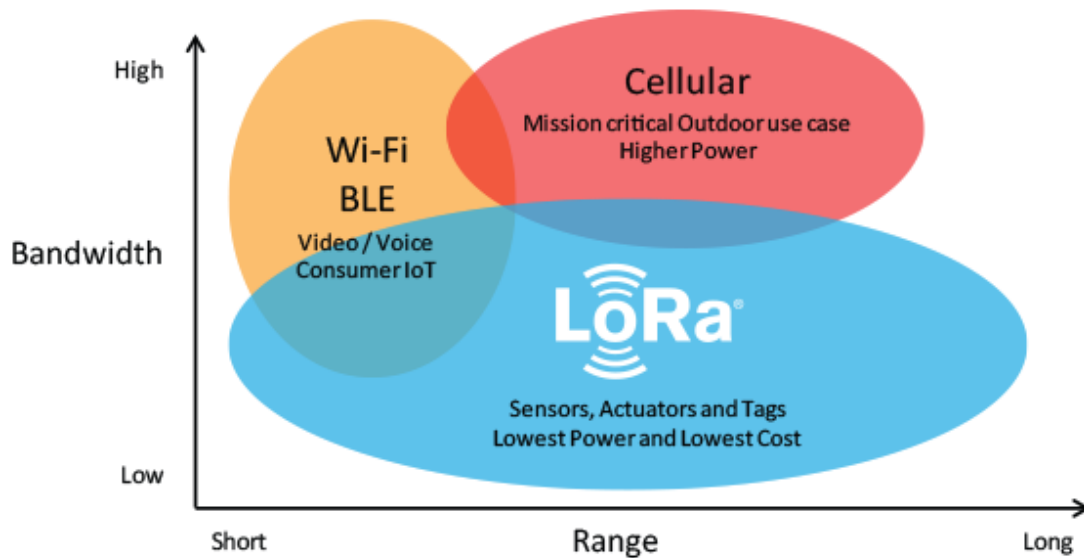


Figure 4.4: Connective Technologies compared against Bandwidth and Range [33]

4.2 Software for Smart Patch

4.2.1 Programming Environment

Arduino IDE, Espressif IDF (IoT Development Framework), and PlatformIO are three different programming environments used for developing software for microcontrollers and embedded systems. Each has its own strengths, weaknesses, and use cases.

	Arduino IDE	Espressif IDF	PlatformIO
Purpose	Beginner-friendly prototyping	Advanced ESP32/ESP8266 development	Cross-platform IoT development
Language Support	C/C++ with simplified syntax	Full C/C++ language support	Supports multiple languages
Hardware Compatibility	Extensive hardware compatibility	Specifically for ESP32/ESP8266	Broad hardware support
Integration	Integrated Development Environment	Integrated toolchain for ESP	Integrated platform and library manager

Table 4.4: Comparative Study between Programming Environments

For the project, Arduino IDE emerges as the preferred choice after comparing the three environments (as outlined in Table 4.4). Its beginner-friendly nature suits the project's prototyping needs, and it offers extensive hardware compatibility. Additionally, Arduino IDE's simplified syntax aligns well with the project's requirements.

4.2.2 Programming Languages

ESP32 is compatible with 2 programming languages - Arduino C++ and MicroPython. Both of these languages are popular and used widely to program the firmware. The table 4.5 provides features of each of these languages.

Features	Arduino C++	MicroPython
Use	Firmware	Firmware
Programming Language	Low-level language with simplified syntax.	Python optimized for micro-controllers.
Abstraction level	Provides abstraction for hardware complexities.	Abstracts hardware with clean Python code.
Ease of Use	Beginner-friendly with a simplified structure.	Beginner-friendly, less complex than C/C++.
Memory Usage	Generally efficient memory usage.	May use more memory compared to C/C++.
Application Scope	Ideal for firmware development in embedded systems.	Suited for microcontroller programming.
Ecosystem	Rich ecosystem, extensive libraries.	Growing ecosystem for microcontrollers.
Concurrency Model	Sequential execution with <code>setup()</code> and <code>loop()</code> .	Supports concurrency but less efficiently.
Frameworks	Arduino framework for embedded systems.	Limited frameworks for microcontrollers.
Resource Efficiency	Generally efficient in resource usage.	May consume more resources.

Table 4.5: Comparative Study between Arduino C++ and MicroPython

Both programming languages demonstrate efficiency in their respective ways. To determine the most suitable choice for the project, a decision matrix is employed (shown in Table 4.6). Each crucial feature is assessed for both languages, assigning a weighted score to each feature. Both scores and weights are provided in a range of 1 to 5.

Features	Weight	Arduino C++		MicroPython	
		Score	Weighted Score	Score	Weighted Score
Familiarity	3	2	6	3	9
Ease of Use	3	3	9	4	12
Memory	4	4	16	3	12
Ecosystem	3	4	12	3	9
Total			43		42

Table 4.6: Decision Matrix for Programming Languages

Following the completion of the decision matrix, it is determined that Arduino C++ emerges

as the more fitting choice of programming language for the project.

4.3 Software for Web Dashboard

The dashboard would be nothing but a website made using HTML, CSS and Javascript (JS). The programming environment of choice will be Visual Studio Code (VSCode), providing a robust platform for web development.

The Flask framework will be utilized for developing the backend framework of the web dashboard. Flask is a lightweight and flexible micro-framework for Python. It offers simplicity and ease of use, making it suitable for building web applications.

Additionally, a mixture of SQLite (for testing) and MySQL (for production) databases will be employed to manage data storage and retrieval efficiently.

1. SQLite offers a lightweight and self-contained database solution, ideal for testing and development environments. Its simplicity and zero-configuration setup streamline the process of data storage and retrieval during development.
2. MySQL is a robust relational database management system, is well-suited for production environments due to its scalability, performance, and reliability. It provides advanced features for managing large datasets and ensuring data integrity for the web dashboard in a production setting.

4.4 Messaging API: Twilio API

The Twilio API provides a robust platform for integrating messaging capabilities into IoT Devices. With Twilio, developers can send and receive SMS, MMS, and even WhatsApp messages programmatically, enabling real-time communication with users. Twilio API offers versatile messaging channels, global coverage, developer-friendly API, scalability, reliability, and security. This makes it the ideal choice for integrating seamless and efficient communication capabilities into the web dashboard.

Chapter 5

Design

The Design chapter talks about the architectural blueprint of VitalMonitor. This includes an overview of the system’s architecture, followed by an in-depth analysis of each component’s design and role within the larger ecosystem. From the setting up of the database to the intricate firmware running on each smart patch, the design choices made across VitalMonitor will be explored to elucidate their contributions to the system’s overall functionality, reliability, and scalability. This chapter is written in accordance with v8 iteration of the firmware.

Additionally, the chapter illustrates the thoughtful considerations behind the chosen architectural patterns, dissects the interactions between system components, and explains the rationale behind the design decisions. This chapter sets the stage for understanding how each element of VitalMonitor contributes to a cohesive and robust system.

For simplicity, the project uses HCP(s), user(s), and wearer(s) interchangeably. Similarly, the terms Smart Patch(es), device(s) and IoT device(s) are used interchangeably.

5.1 System Design

5.1.1 Design Approach

The system design of VitalMonitor integrates multiple architectural principles, deviating from traditional approaches to create a tailored solution. It is structured into three primary layers: Presentation, Application, and Data Layer.

- **Presentation Layer:** The web dashboard serves as the presentation layer, providing a user interface (UI) for users to interact with the system. This layer presents data collected from IoT devices in a visual and understandable format through the dashboard.
- **Application Layer:** The middleware component serves as the application layer, handling the business logic of receiving, and distributing data from Smart Patches. It acts

as an intermediary between the presentation layer and the data layer, performing tasks such as data aggregation and routing.

- **Data Layer:** The Smart Patches represent the data layer, where data is generated and collected from the real world. This raw data from sensors is then processed and transmitted to the middleware for further processing and storage.

Communication between these layers primarily occurs through APIs, designed following RESTful principles to promote simplicity and flexibility.

Moreover, the Smart Patch logic adopts a hybrid architecture:

- **Edge-Computing:** Edge Computing optimizes data processing by executing computations closer to the data source (here the Smart Patch), effectively reducing latency and conserving bandwidth. This strategy facilitates local data processing, eliminating irrelevant information and transmitting only essential insights to the cloud. Urgent actions, like sending emergency SMS via Twilio API, are executed swiftly at the edge to minimize delays.
- **Cloud-Based:** The cloud serves dual purposes: firstly, as a data repository, and secondly, for processing incoming data from devices. Settings originating from the Presentation layer are forwarded to Smart Patches from the cloud.

This design rationale was chosen to:

1. Enhance flexibility in development and deployment processes.
2. Facilitate continuous integration and delivery practices.
3. Inherently support a distributed data processing model, crucial for IoT environments.

5.1.2 Component Roles and Ideology

VitalMonitor operates on a coordinated data flow mechanism, where each component plays a critical role. Each component is designed with a specific role in mind as per the above design choices. This systematic interaction ensures seamless end-to-end data flow, from data capture to user-level presentation. The components are:

- The Dashboard is the user interface through which users interact with the system. It is designed to offer intuitive controls and insightful analytics.
- The Smart Patch is a component of VitalMonitor, equipped with sensors and a micro-controller to accurately collect and relay vital health data.
- The Middleware acts as the central nervous system, directing the flow of information and ensuring the components of VitalMonitor communicate effectively.

5.1.3 Data Flow Architecture

Figure 5.1 illustrates the interconnectivity and the specific pathways through which data travels within the VitalMonitor system. This diagram provides a visual representation of the sequential and logical flow of data, from the initial collection at the smart patch, through processing by the middleware, to its final storage and presentation in the dashboard. Figure 5.2 goes further into how a Smart Patch gets data from sensors and integrates with Twilio API and Adafruit IO. It exemplifies how the design ensures not only the efficient management of data but also its security and integrity as it moves through various system components.

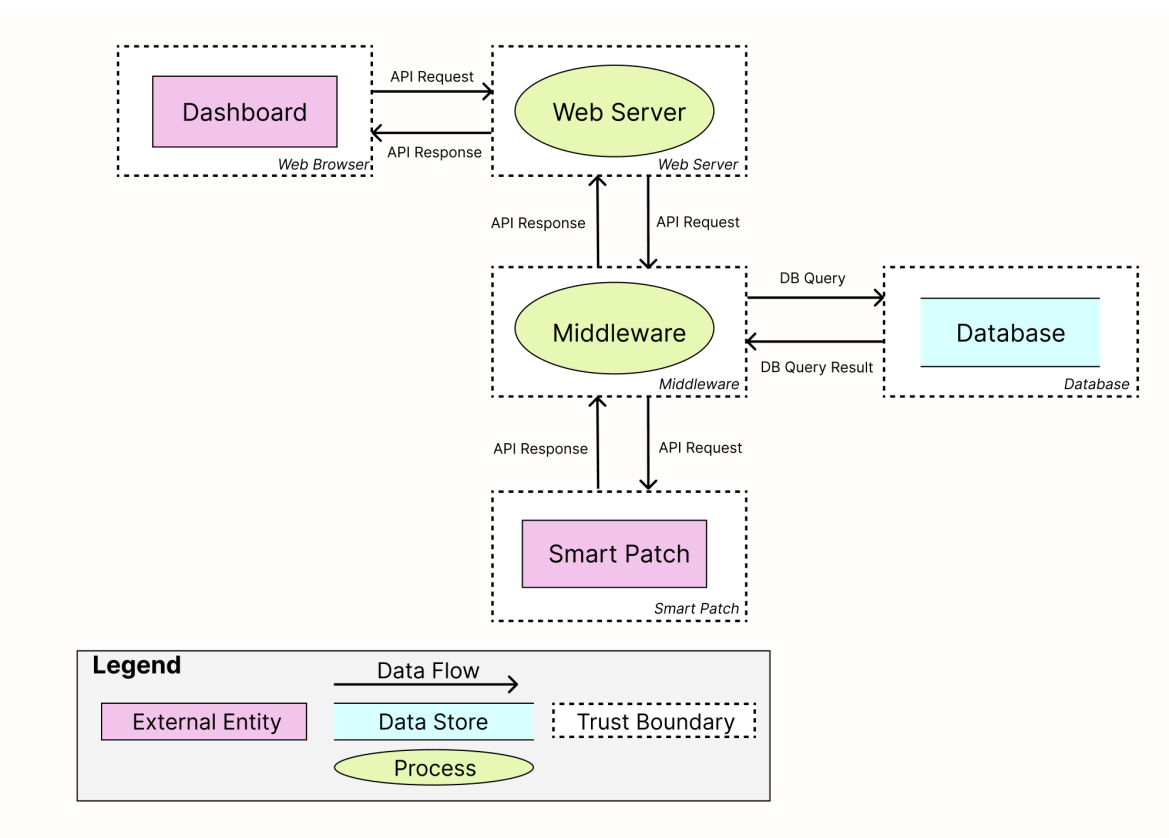


Figure 5.1: Data Flow Diagram 1

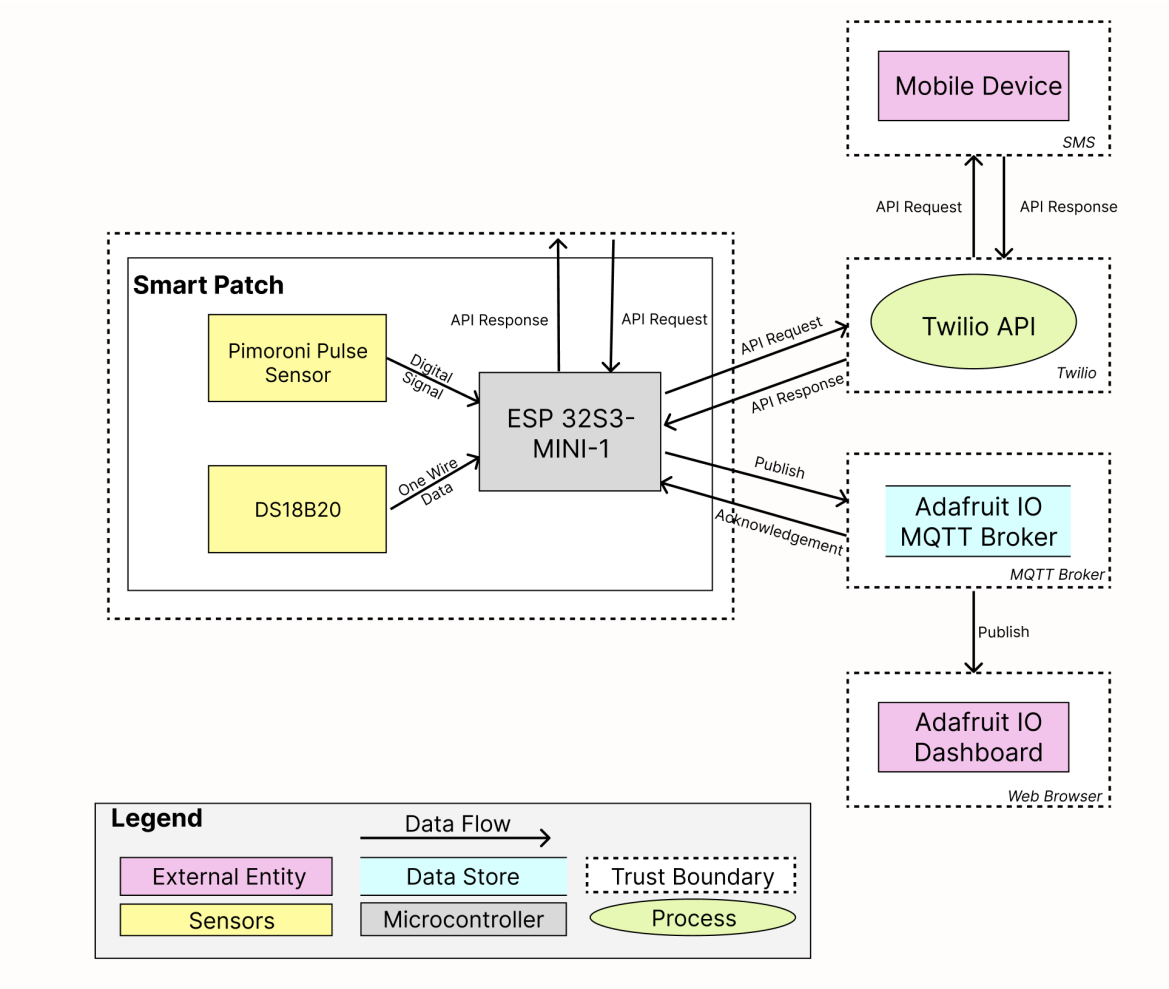


Figure 5.2: Data Flow Diagram 2

5.2 Database Design

The database is designed using MySQL and is hosted on Google Cloud, leveraging the platform’s scalable infrastructure to handle the data management needs of the IoT ecosystem. With a focus on relational integrity and query efficiency, the database schema is structured to support the platform’s core functionalities: user management, device tracking, and real-time data analytics.

Figure 5.3 provides a visual representation of the relational models and their interconnections. It illustrates the foreign key relationships and cardinality between tables, reflecting the database’s normalized design that promotes data integrity and reduces redundancy.

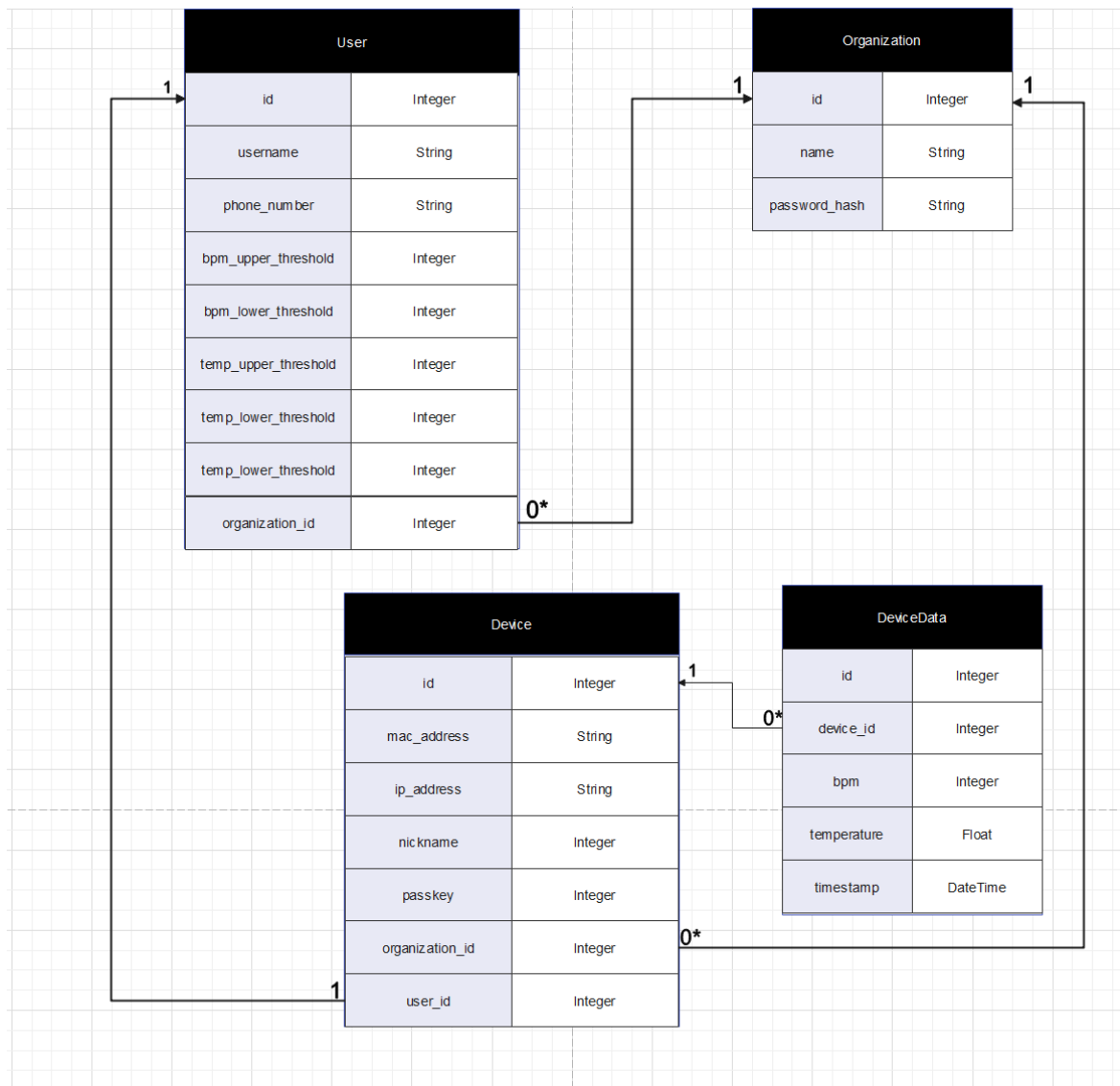


Figure 5.3: Database Diagram

The main entities of the database are:

5.2.1 Organizations

Organizations form the top hierarchical level within VitalMonitor, representing entities that manage a collection of users and devices. The organization's model features fields for identification, authentication, and relationships to devices and users. Robust security measures are implemented, including hashed passwords using bcrypt to safeguard access credentials.

5.2.2 Devices

The Device model is a representation of physical IoT devices in the system, uniquely identified by a MAC address. Devices are associated with an organization and optionally linked to a specific user. Each device holds a passkey for secure communications and maintains a record of health data streams through its one-to-many relationship with the DeviceData model.

5.2.3 Users

Users are associated with an organization and have a direct connection to a single device. The User model contains contact information and configurable thresholds for health metrics, enabling personalized monitoring and alerts.

5.2.4 DeviceData

The DeviceData model captures the essence of IoT functionality within VitalMonitor. It stores time-series data from IoT devices, including heart rate (bpm) and temperature readings, with precise timestamps. This model ensures that vital health data is recorded, enabling historical data analysis and real-time monitoring.

The database's design is crafted to be intuitive and efficient, facilitating straightforward interactions for the system's backend services. By adopting this structured approach, VitalMonitor ensures fast retrieval of data, which is paramount for real-time applications, while also maintaining the flexibility to scale and adapt to future requirements.

The firmware is designed to handle the exchange of threshold parameters and user information with the middleware. If the measured bpm or temperature exceeds the set thresholds, the device utilizes the Twilio API to dispatch an SMS to a designated emergency contact.

5.3 Smart Patch (Firmware) Design

The firmware for Smart Patch is an ESP32S3-MINI-1 microcontroller. It constitutes the operational bedrock for interfacing with the hardware sensors and executing the device's logic. It is meticulously programmed to manage sensor data acquisition, device control, network communication, and emergency alerting functionalities.

5.3.1 Hardware Components

The device is equipped with a DS18B20 temperature sensor and a Pimoroni Pulse Sensor, tasked with measuring the user's body temperature and heart rate, respectively. The temperature sensor circuit includes a $4.3\text{k}\Omega$ pull-up resistor essential for accurate signal readings. Similarly, a $1\text{k}\Omega$ resistor is employed in the LED circuit to indicate the device's operational status. Activation of the sensors and publication of data is controlled by a user-operated

push button which toggles the data transmission process, with an LED serving as a visual indicator for the current state—lit when data is being sent and unlit otherwise. The device is powered by a LiPo battery, ensuring portability and convenience.

5.3.2 Hardware Circuit Diagram

A detailed circuit diagram is shown in figure 5.4 outlines the wiring and connections of all components. This showcases the precise electrical relationships and configurations necessary for optimal performance. This diagram serves as a guide for the physical assembly of the hardware components. In the diagram 3V connection is shown using a red wire and ground (or GND) connection is shown using a blue wire. Legend to this figure is provided in figure 5.5.

Adafruit IO Dashboard is also designed to get data directly from the firmware for testing purposes. This dashboard allows lightweight transfer of data and allows visualisation (eg., time-series graph) of the health data - which is useful to see how efficiently the data is read by the sensors. Check Appendix D for more details on Adafruit IO Dashboard and how it works.

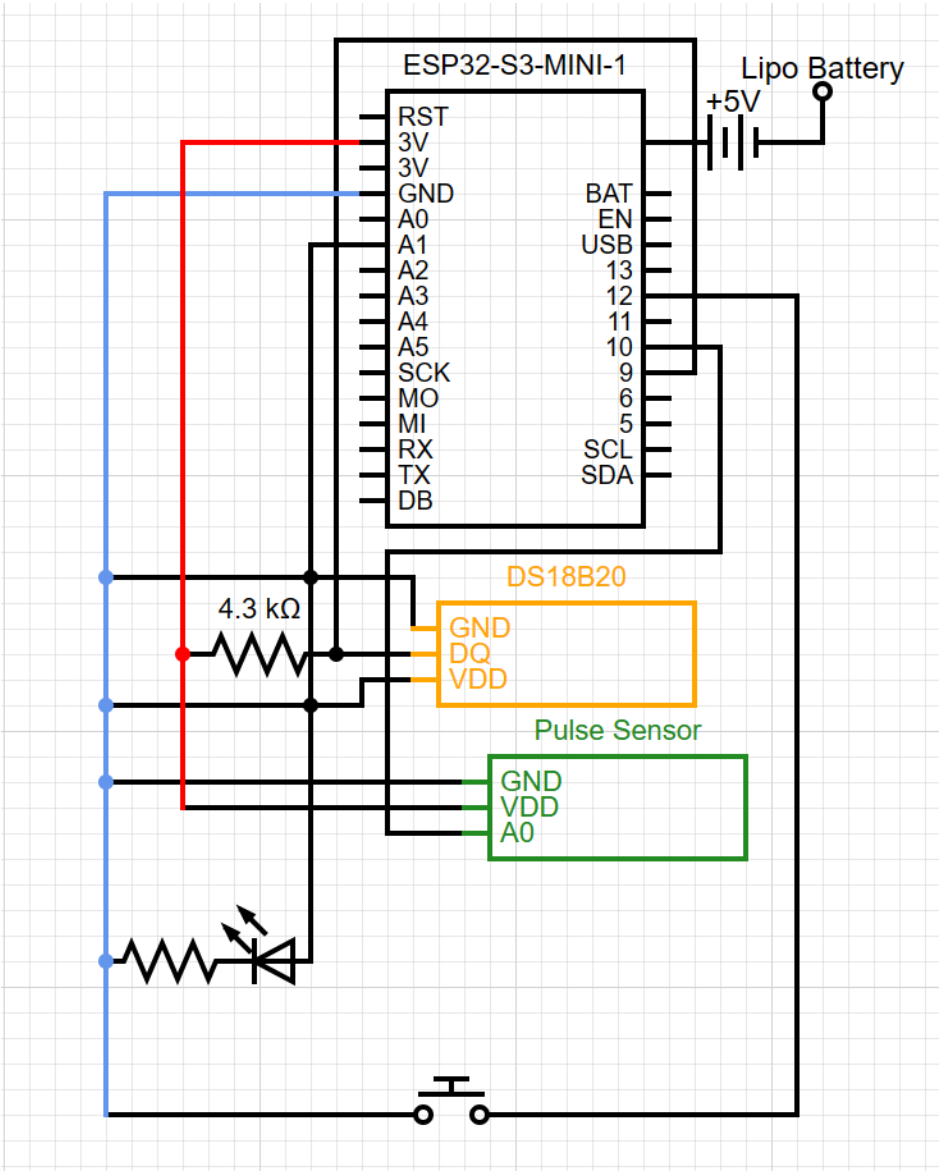


Figure 5.4: Circuit Diagram of Smart Patch

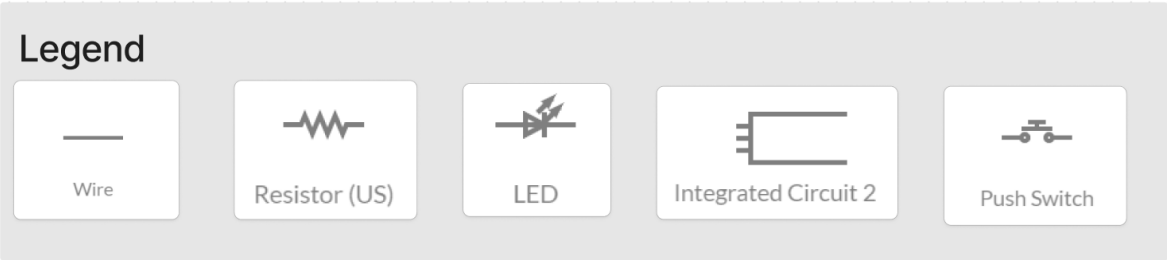


Figure 5.5: Legend for circuit diagram

5.4 Middleware Design

The Middleware within VitalMonitor serves as the central communication hub, orchestrating interactions among the platform's components. It acts as the pivotal component that integrates dashboards, databases, and smart patches, ensuring seamless functionality and cohesion.

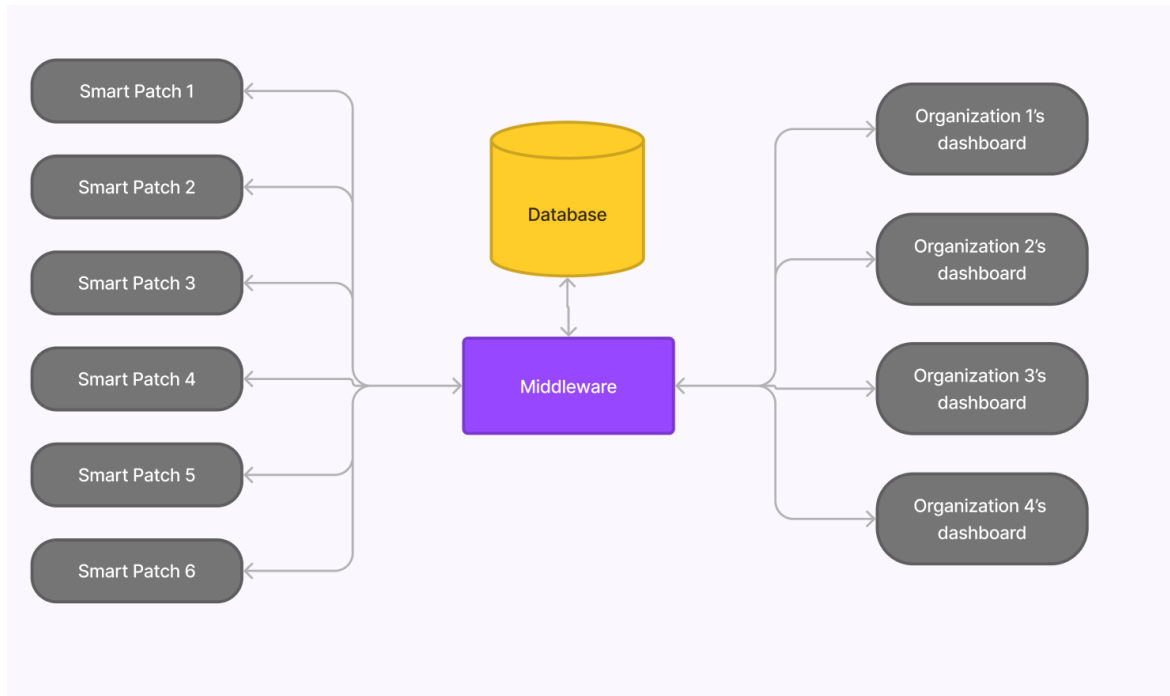


Figure 5.6: Middleware

5.4.1 Architecture and Responsibility

The Middleware, designed as an Application Server Middleware (ASM), bears the responsibility of overseeing vital functions necessary for the optimal operation of the VitalMonitor platform. It shoulders tasks such as processing requests, enforcing business logic, managing data, and facilitating communication between the different components of the system.

Additionally, following a microservices-like architecture, the Middleware offers four primary services to the system:

- Session Management Service
- User and Device Management Service
- Data Ingestion Service

- Data Retrieval Service

This is done in such a way because when the load of number of devices and dashboard increases, the middleware will not be able to handle it. There fore, in future, isolating the service and having them as their own services along with a load balancer, this allows more controlled and managed data.

5.4.2 Design Principles

Several key principles guided the design of the Middleware:

1. **Communication Hub:** The Middleware is designed to be a central communication hub. It is responsible for the uninterrupted flow of data between devices and the user interface. It acts as the API Gateway, receiving API calls from the dashboard, interpreting them, and routing these to the appropriate services or data stores.
2. **Abstraction:** The Middleware is designed to provide a level of abstraction that simplifies interactions between the user interface and the system's backend. It abstracts the complexities of the underlying database structures and device communication protocols from the end-user, presenting a simple and coherent interface to the dashboard and using of smart patch.
3. **Data Management:** The Middleware is designed to provide robust data management. It provides a secure and efficient gateway to the database, handling transactions, query processing, and data synchronization. This ensures data integrity and consistent state across the platform, even as it scales.
4. **Unified Connection for Multiple Entities:** The Middleware is designed with a comprehensive routing and authentication system. This facilitates concurrent connections across multiple dashboards and smart patches. It supports a multiplexed network structure where each entity is uniquely identified and managed, maintaining a coherent state across the ecosystem.

By adhering to these principles, the Middleware embodies a pivotal component within the VitalMonitor ecosystem - fostering efficient communication, seamless operation, and scalable growth.

5.5 Dashboard Design

The VitalMonitor dashboard is specifically designed to serve as a centralized control panel for organization administrators, enabling them to efficiently manage healthcare professionals' data. This platform facilitates real-time monitoring and management of user profiles and devices, allowing admins to set health thresholds, add or delete users, and assign devices. The dashboard is crafted to cater to the specific needs of health organization administrators, providing them with intuitive tools to enhance operational efficiency and patient care.

5.5.1 Technology and Tools

The dashboard is designed using Flask, a micro web framework that offers flexibility and simplicity for server-side logic. Flask's lightweight and modular nature makes it an ideal choice for constructing a responsive and efficient web application. For the frontend, HTML, CSS, and JavaScript are employed to create a dynamic and interactive user interface that responds to the administrators' operational needs effectively.

5.5.2 Design Pattern

Decorator Pattern

The use of the Decorator Pattern is fundamental in the design of our Flask application, especially in managing route associations and enhancing function capabilities without modifying their core responsibilities:

- **Routing:** Utilizing Flask's `@app.route()` decorator simplifies the mapping of HTTP requests to Python functions, making the association between URL routes and server-side logic clear and maintainable.
- **Custom Decorators:** Custom decorators are designed to manage user session and authentication checks. These intercept and verify requests to ensure user authentication before accessing specific functionalities, thus enhancing security and user management efficiency.

Model View Controller-like Structure

While Flask does not enforce an MVC architecture, the design of the dashboard subtly incorporates elements of this pattern, enhancing clarity and separation within the application:

- **Model (M):** The business logic and data interactions are handled in the background, interfacing with the middleware. This includes user and device management functionalities, where data integrity and operations like add, update, and delete are executed. Middleware provides access to requested data but not the full model and its data directly, for an additional layer of security.
- **View (V):** The presentation layer is managed through Flask's template system. HTML templates are rendered based on the data processed by the controllers, providing a user-friendly interface for interaction. Templates for pages like *add_device.html* and *user_page.html* define how data is presented and ensure a consistent user experience.
- **Controller (C):** Flask routes act as controllers, orchestrating the application's response to user inputs and interactions. They process incoming requests, call the appropriate model functions, and determine the correct views to render, completing the user request lifecycle.

5.5.3 User Flow

The user flow diagram (Figure 5.7) visually represents the navigational pathway administrators take through the dashboard. It highlights the sequence of interactions from login to detailed device and user management.

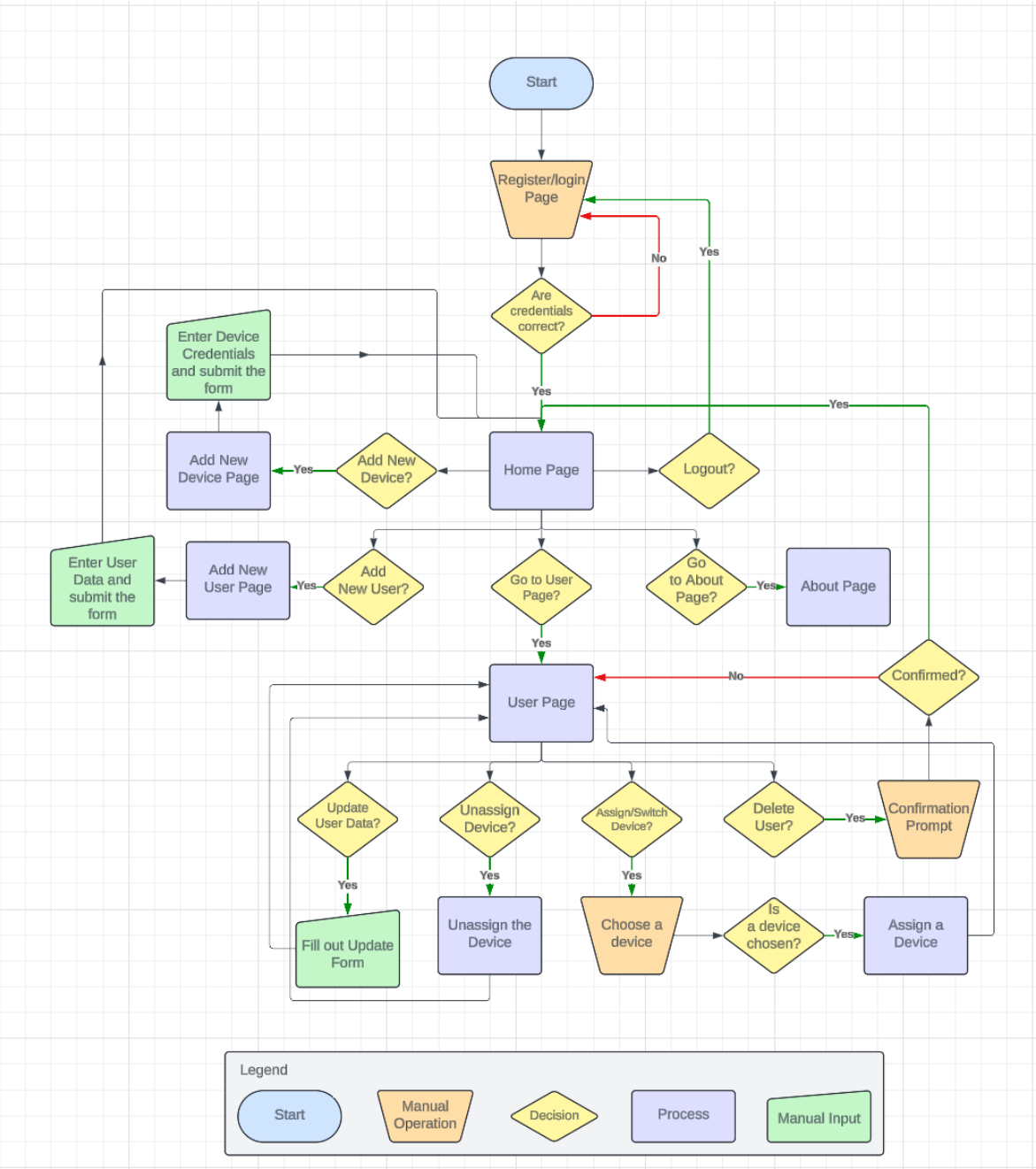


Figure 5.7: User Flow Diagram for Dashboard

5.5.4 Wireframes

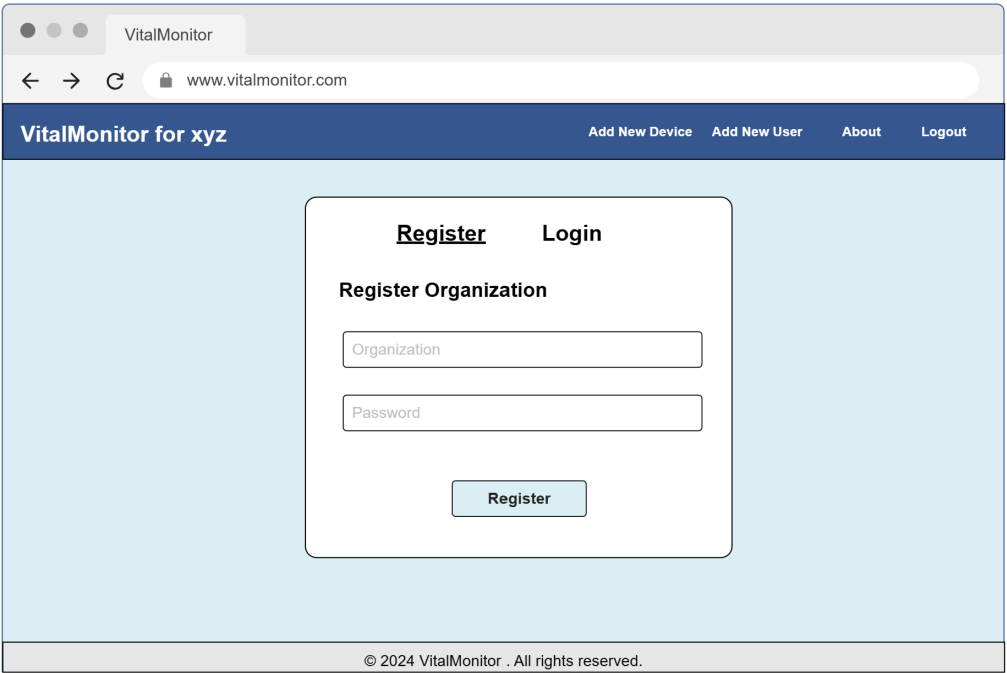


Figure 5.8: Landing Page to register/login

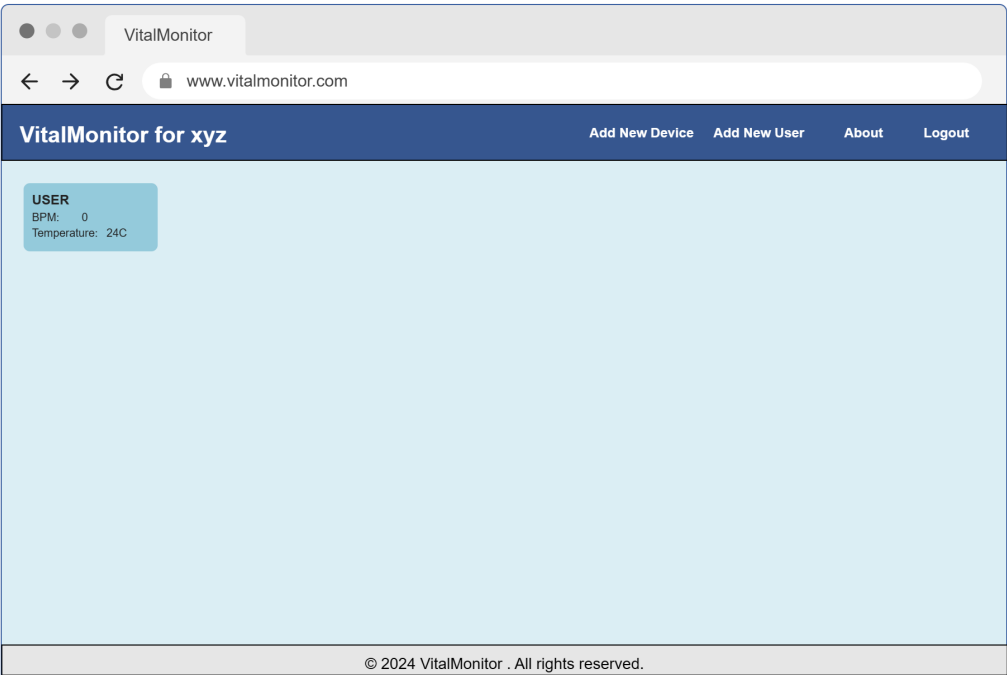


Figure 5.9: Home Page

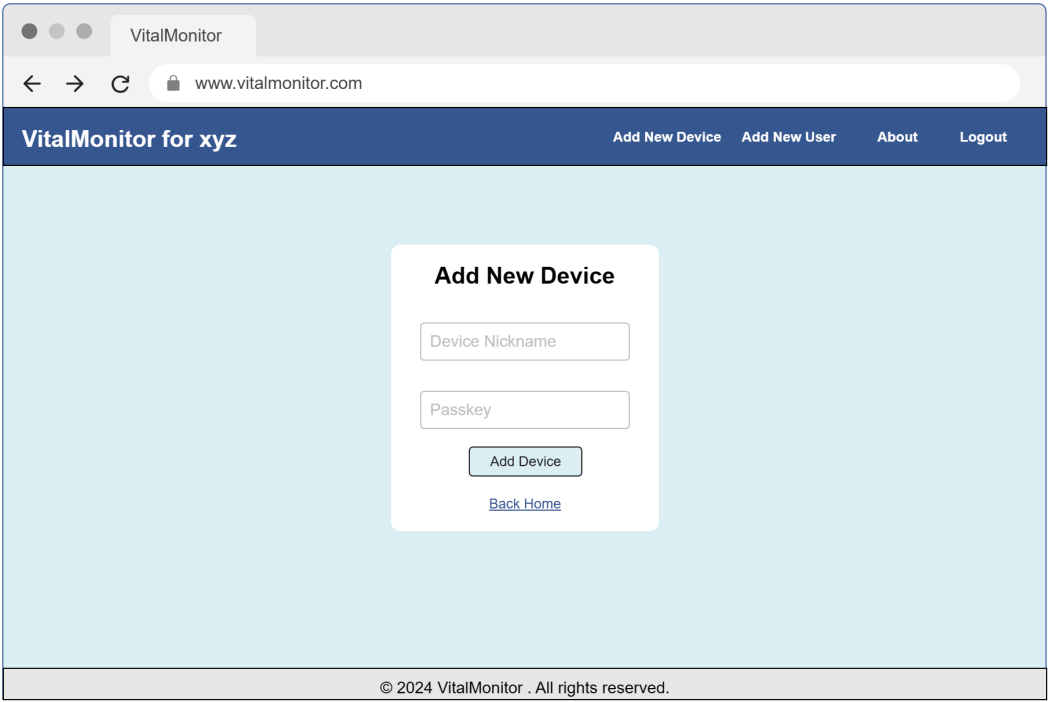


Figure 5.10: Add New Device page

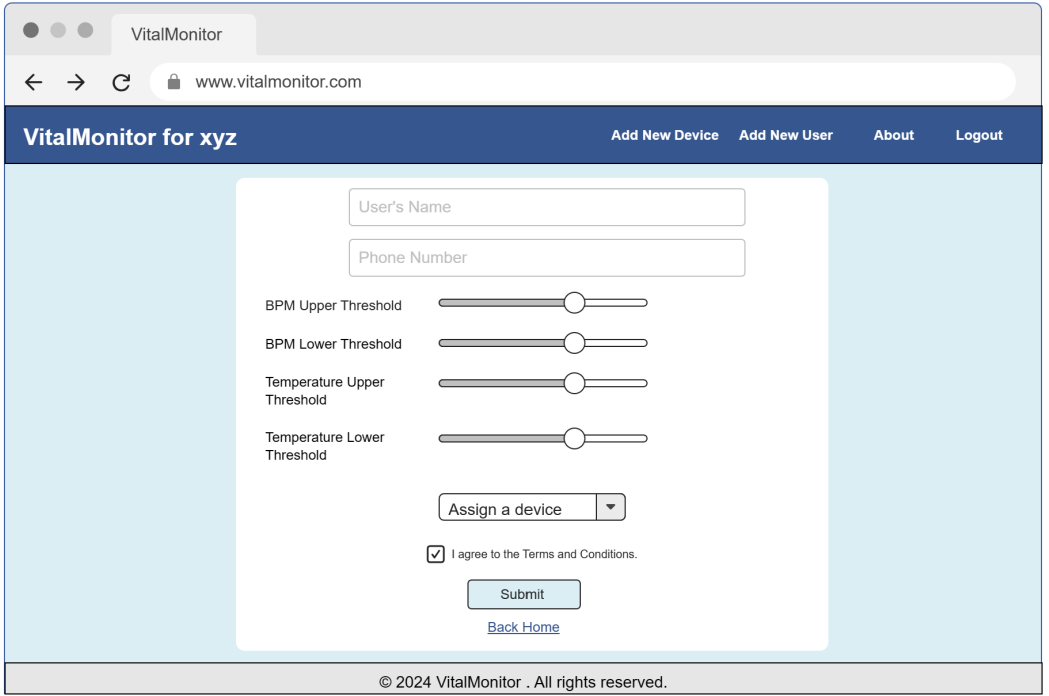


Figure 5.11: Add New User Page

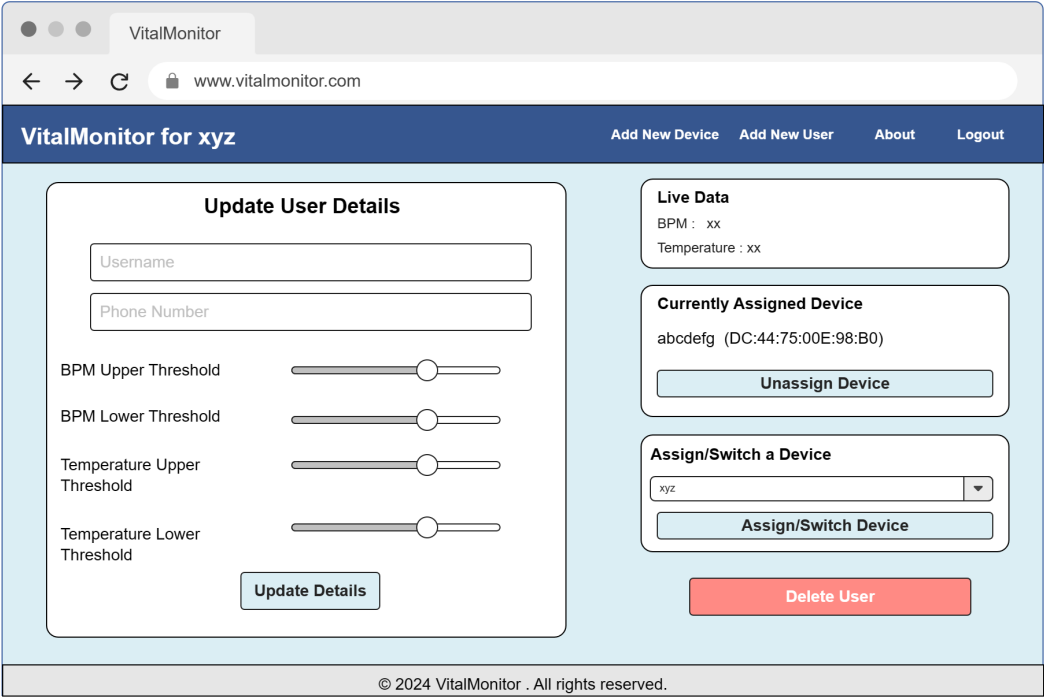


Figure 5.12: User Page

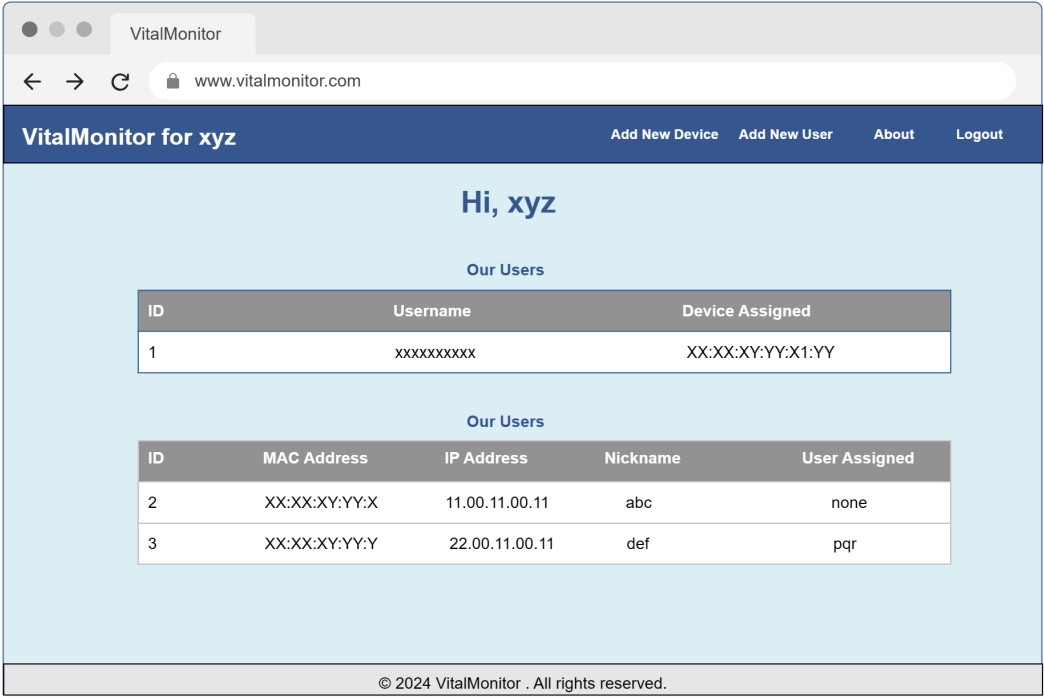


Figure 5.13: About Page

5.6 Network Communication and API Design

VitalMonitor's networking layer is designed to enable robust and efficient interactions between front-end applications, back-end services, and IoT devices. By utilizing RESTful principles and leveraging the JSON data format, the system ensures streamlined data exchange and functionality that scales effectively with the demands of healthcare monitoring. Along with it, Smart Patches are designed to create their own Asynchronous Web Servers to facilitate transmission of data. Additionally, they are designed to use MQTT Protocol to publish data on Adafruit IO Dashboard.

5.6.1 API and API Endpoints

The API endpoints of VitalMonitor are meticulously designed to reflect a resource-oriented approach, enabling straightforward CRUD operations. Here's a closer look at the design:

- **Structured for Simplicity and Efficiency:** Each endpoint is crafted to represent and manipulate a distinct entity, such as users or devices, simplifying interactions and reducing complexity.
- **Consistent Data Format:** The system exclusively use JSON for all exchanges, ensuring compatibility and ease of development across various platforms.
- **Stateless Operations:** Ensuring each request is self-contained, the APIs maintains statelessness, which is key to enhancing the scalability and reliability of distributed systems.

There are multiple API endpoints for each of the components of the system :

Middleware:

Table 5.1: API Endpoints for Middleware

Endpoint	Method	Description
/register	POST	Register a new organization with name and password.
/login	POST	Authenticate organization credentials.
/add-new-user	POST	Add a new user to an organization, specifying necessary personal and health threshold details.

Continued on next page

Table 5.1 – Continued from previous page

Endpoint	Method	Description
/add-new-device	POST	Add or update a device in the system, specifying details like MAC address and IP address.
/associate-device	POST	Associate a device with an organization using device nickname and passkey.
/assign-device-to-user	POST	Assign a device to a specific user, linking it with user ID and device ID.
/unassign-device/{device_id}/{user_id}	POST	Unassign a device from a user, removing all associated user details from the device.
/update-user-details	POST	Update details for an existing user, including contact information and health thresholds.
/get-device-data/{device_id}	GET	Stream real-time data from a specific device.
/save-device-data	POST	Save data received from a device to the database.
/delete-device/{device_id}	DELETE	Remove a device and all its associated data from the system.
/delete-user/{user_id}	DELETE	Remove a user and all their associated details from the system.
/delete-organization/{organization_id}	DELETE	Remove an entire organization and all associated users, devices, and data.
/devices	GET	Retrieve all devices associated with an organization.
/users	GET	Retrieve all users within an organization.
/user/{user_id}	GET	Retrieve detailed information about a specific user.

Dashboard:

Table 5.2: Dashboard API Endpoints

Endpoint	Method	Description
/register	POST	Registers and logs in a new organization.
/login	POST	Authenticates and sets session for an organization.
/	GET	Displays the main dashboard if logged in.
/register_login	GET	Shows registration and login portal.
/logout	GET	Logs out and clears session.

Continued on next page

Table 5.2 – Continued from previous page

Endpoint	Method	Description
/add-device-page	GET	Shows the add device page for logged-in users.
/add-device	POST	Adds a new device and displays status message.
/submit-user-details	GET	Displays unassigned devices for user assignment.
/add-user	POST	Adds a new user and optionally assigns a device.
/user/user_id	GET, POST	Allows updating user details and device assignment.
/device/device_id/update-details	POST	Sends updated user data to a device.
/sendDetails	POST	Receives and updates live BPM and temperature data.
/data-stream	GET	Streams live BPM and temperature data.
/about-us	GET	Displays organization details for logged-in users.

Smart Patch:

Table 5.3: Smart Patch API Endpoints

Endpoint	Method	Description
/receive-user-details	POST	Receives and stores user details for device assignment, enabling personalized data monitoring and alerts.
/clear-user-details	POST	Clears all user-specific details from the device, effectively unassigning it from a user.
/stop-publishing	GET	Handles request to stop data publishing, which could be triggered remotely via web request.

Lastly, feeds will be designed as per MQTT Protocol for sending data to Adafruit IO Dashboard. See Appendix D for more information.

5.6.2 Error Handling and Management

The implemented networking layer should be able to handle different HTTP client and server errors that can occur. The supported errors are:

Error Code	Error Descriptions
400	Bad Request
401	Unauthorised
404	Not Found
409	Conflict with current state/resources
500	Network Errors

Table 5.4: Supported Error Codes

More error conditions can be added in future, if deemed necessary. However, these cover the minimum needed to provide a reliable user experience.

Chapter 6

Implementation and Testing

6.1 Implementation and Testing Strategy

The implementation of the VitalMonitor system was orchestrated using Agile methodologies to foster iterative development and continuous integration across all subsystems: the Smart Patch, the Middleware application server, and the Dashboard web application. Beginning with the Smart Patch, the development strategy emphasized incremental builds, each enhancing specific functionalities such as sensor integration, WiFi connectivity, Adafruit IO, and webserver capabilities, with rigorous assessment through a detailed test suite that evaluated each function against predefined test specifications. The development of Smart Patch was divided into 8 iterations or versions - each either adding a new functionality or solving a previous problem.

Following the initial development, the Dashboard was implemented using Flask, HTML, CSS, and JavaScript, supported by an SQLite database for local testing before transitioning to a full integration with the Middleware. This Middleware, also Flask-based, served as the central communication hub, facilitating robust data interactions between the Smart Patch, Dashboard, and a MySQL database hosted on Google Cloud. Applying a consistent test suite approach across all subsystems ensured high standards of functionality and performance were maintained, leading to a seamlessly functioning integrated system upon final assembly.

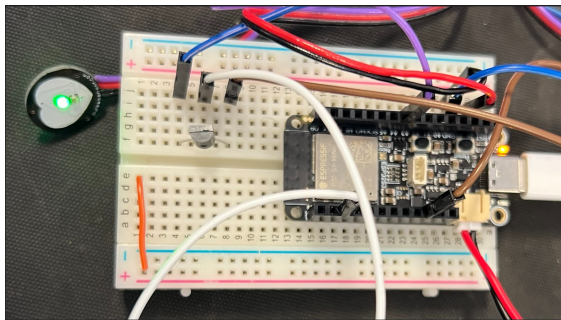
6.2 Firmware

The development of the Smart Patch firmware was characterized by a series of iterative improvements aimed at enhancing sensor accuracy, data handling, and user interaction. The project began with basic analog signal processing and gradually incorporated more sophisticated digital technologies and cloud-based integrations. Each iteration was driven by the dual goals of improving data reliability and enhancing user experience, while also adapting to technical challenges encountered along the way.

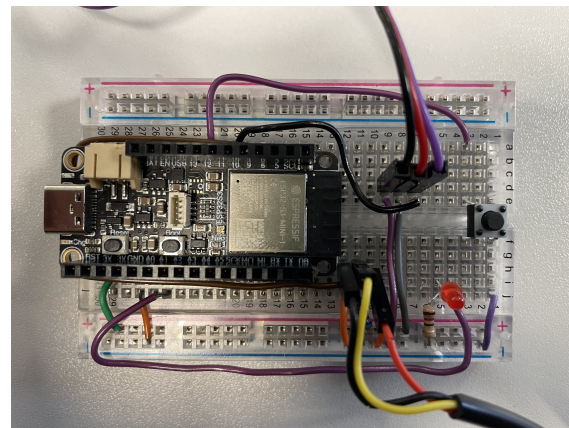
Initially, the firmware relied on analog signals to gather data from pulse sensor and LM-35 temperature sensor, which resulted in unstable and inaccurate readings. The shift to digital signal processing, and later to digital sensors like the DS18B20 for temperature sensing, marked significant advancements in data accuracy and stability. Integrating the system with cloud platforms such as Adafruit IO and external APIs like Twilio for SMS notifications brought about real-time data monitoring and interactive functionalities, further extending the capabilities of the Smart Patch.

The iterative development process was guided by continuous testing and feedback, which helped identify and rectify shortcomings in real-time. Challenges such as managing non-static IP addresses were addressed through architectural adjustments, notably the introduction of middleware to maintain connectivity and manageability. The final iterations focused on user management and data publishing control, allowing for more flexible and user-centric operation.

As of for the physical hardware - majorly the changes can be seen in from v1 to v6. Rest the hardware remained same. These can be seen in figures 6.1a and 6.1b.



(a) V1 Firmware



(b) V6 Firmware

Figure 6.1: Comparison between V1 and V6 Firmware

Table 6.1 provides a summary of each iteration.

This section will discuss details as per the eight iteration.

Iteration	Description	Key Changes	Outcome
v1	Initial implementation using analog signals.	Utilized PulseSensor and LM-35 sensors with analog outputs.	Data was unstable and lacked accuracy.
v2	Enhanced data handling by discarding outliers.	Implemented data smoothing by removing the top and bottom 10% of readings; maintained analog LM-35 while shifting PulseSensor to digital.	Improved pulse data slightly; temperature readings remained unreliable.
v3	Switch to digital temperature sensor.	Replaced LM-35 with DS18B20 for temperature measurements.	Achieved highly accurate and stable temperature data.
v4	Integration with cloud and real-time data dashboard.	Connected to Wi-Fi; established MQTT client for Adafruit IO dashboard.	Successfully reflected real-time data on the dashboard.
v5	Added SMS notifications for temperature alerts.	Integrated Twilio API to send SMS alerts when temperature exceeds 35°C; included rate-limiting.	Reduced spam with effective notification for critical temperature conditions.
v6	Codebase optimization and user data management.	Refactored code into separate files; added web interface for user data input.	Enhanced efficiency and usability with better organization and user data integration.
v7	Implemented data smoothing and reliability checks.	Introduced moving average logic for data smoothing and set tolerance thresholds for sensor data.	Further stabilized sensor data, but faced challenges with dynamic IP addresses.
v8	Addressed connectivity issues; enhanced user management.	Implemented middleware for IP management; enhanced user-centric functionality and data privacy.	Resolved IP issues and improved system flexibility for user management and data handling.

Table 6.1: Summary of Firmware Iterations

6.2.1 Sensor Integration

The integration of sensors into the firmware of the Smart Patch was a critical component of its development, enabling the device to capture real-time physiological data with high accuracy. This subsection details the integration processes for both the Pimoroni Pulse Sensor and the DS18B20 Temperature Sensor, as well as the implementation of a moving average filter to enhance sensor accuracy.

Integration of Pimoroni Pulse Sensor

The integration of the Pimoroni Pulse Sensor was implemented using the PulseSensorPlayground library to facilitate the capture of heartbeat signals. Configuration involved setting the input pin and threshold for beat detection. Listing 6.1 is the code used in the file `/esp/config.h`:

```
1 // Pulse Sensor Setup
2 const int PulseWire = 10;
3 int Threshold = 550;
4 PulseSensorPlayground pulseSensor;
5
6 void setupSensors() {
7     pulseSensor.analogInput(PulseWire);
8     pulseSensor.setThreshold(Threshold);
9     if (pulseSensor.begin()) {
10         Serial.println("PulseSensor initialised!");
11     }
12 }
```

Listing 6.1: Code for Pulse Sensor Integration

This setup ensures that the pulse sensor is initialized correctly and ready to transmit pulse rate data to the main application.

Integration of DS18B20 Temperature Sensor

The DS18B20 Temperature Sensor, a digital sensor, was integrated using the OneWire and DallasTemperature libraries to allow for accurate temperature readings. The sensor was configured with the following initialization code (Listing 6.2 from the file `/esp/config.h`):

```
1 // DS18b20 Sensor Setup
2 const int TempWire = 9;
3 OneWire oneWire(TempWire);
4 DallasTemperature tempSensors(&oneWire);
5
6 void setupSensors() {
7     tempSensors.begin();
8 }
```

Listing 6.2: Code for DS18B20 Integration

This configuration initializes the temperature sensor and prepares it to read temperatures in Celsius, which are then processed by the firmware.

Sensor Accuracy - Moving Average

To smooth out fluctuations in the data from both sensors and enhance accuracy, a moving average filter was implemented. This method averages the data points within a defined window to mitigate the effects of transient and noisy data. The following code (Listing 6.3 from the file `/esp/esp.ino`) illustrates the implementation of this moving average for both the pulse and temperature sensors:

```

1  const int movingAverageWindowSize = 10;
2  float bpmSum = 0;
3  std::queue<int> bpmReadings;
4  float bpmMovingAverage = 0;
5  float tempSum = 0;
6  std::queue<float> tempReadings;
7  float tempMovingAverage = 0;
8
9  // Tolerance constants
10 const int BPM_TOLERANCE = 10; // Adjust based on testing and sensor accuracy
11 const float TEMP_TOLERANCE = 0.5; // Adjust based on testing and sensor
    accuracy
12
13 void updateMovingAverage(int bpm, float tempC) {
14     // Update BPM moving average
15     if (bpmReadings.size() >= movingAverageWindowSize) {
16         bpmSum -= bpmReadings.front();
17         bpmReadings.pop();
18     }
19     bpmSum += bpm;
20     bpmReadings.push(bpm);
21     bpmMovingAverage = bpmSum / bpmReadings.size();
22
23     // Update Temperature moving average
24     if (tempReadings.size() >= movingAverageWindowSize) {
25         tempSum -= tempReadings.front();
26         tempReadings.pop();
27     }
28     tempSum += tempC;
29     tempReadings.push(tempC);
30     tempMovingAverage = tempSum / tempReadings.size();
31 }

```

Listing 6.3: Implementation of Moving Average to increase sensor accuracy

Through this implementation, both pulse and temperature readings are consistently averaged out, providing more reliable data to the system for further processing and decision-making.

6.2.2 MQTT and Adafruit IO Integration

The integration of MQTT protocol with Adafruit IO provides a robust testing solution for remote data monitoring and device management in the Smart Patch system. MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol ideal for IoT applications due to its minimal bandwidth usage and effective data transmission even in unreliable networks.

Configuration and Initialization

The integration involves setting up an MQTT client with Adafruit IO as the broker. This setup utilizes the Adafruit MQTT library to establish a connection and publish data to specific feeds configured in the Adafruit IO dashboard. Listing 6.4 is the primary configuration used to initialize the MQTT client and data feeds.

```
1 #include "Adafruit_MQTT.h"
2 #include "Adafruit_MQTT_Client.h"
3
4 #define AIO_SERVER "io.adafruit.com"
5 #define AIO_SERVERPORT 1883
6 #define AIO_USERNAME "your_username"
7 #define AIO_KEY "your_aio_key"
8
9 // Setup the MQTT client class
10 Adafruit_MQTT_Client mqtt(&espClient, AIO_SERVER, AIO_SERVERPORT,
    AIO_USERNAME, AIO_KEY);
11 Adafruit_MQTT_Publish bpmFeed = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME "/"
    feeds/BPM");
12 Adafruit_MQTT_Publish temperatureFeed = Adafruit_MQTT_Publish(&mqtt,
    AIO_USERNAME "/feeds/Temperature");
```

Listing 6.4: MQTT and Adafruit IO Client Configuration

In this setup, the MQTT client is configured with server details and user credentials. Feeds for BPM and temperature data are defined using the username and a specific endpoint path that corresponds to each data type.

These code snippets and explanations outline the critical aspects of the MQTT and Adafruit IO integration within the Smart Patch system, demonstrating the implementation of network communication protocols to enhance IoT capabilities.

6.2.3 Twilio Integration

Twilio's cloud communication platform was integrated into the Smart Patch system to enable SMS notifications based on specific health data thresholds. This feature is particularly critical for immediate user notification in case of emergency health conditions.

The integration requires setting up authentication with Twilio's API using the account credentials and a messaging service SID, which are defined as constants. These credentials allow the device to authenticate with Twilio's servers securely and send SMS messages through the API.

Sending SMS

The function 'sendSMS' constructs the request to send an SMS using HTTPS POST. It constructs the URL and body dynamically using string streams and sends the request using the ESP32's HTTPClient library.

```

1 bool sendSMS(const char *body) {
2     std::stringstream url, urlEncodedBody;
3     url << "https://api.twilio.com/2010-04-01/Accounts/" << accountNr << "/"
4     Messages";
5     urlEncodedBody << "MessagingServiceSid=" << messagingServiceSid << "&To="
6     << to.c_str() << "&Body=" << body;
7
8     HTTPClient http;
9     http.begin(url.str().c_str());
10    http.addHeader("Content-Type", "application/x-www-form-urlencoded");
11    http.setAuthorization(accountNr, twilioPassword);
12    int httpCode = http.POST(urlEncodedBody.str().c_str());
13
14    http.end();
15    return httpCode == 201; // HTTP 201 indicates Created
16 }

```

Listing 6.5: Function to Send SMS via Twilio

Threshold Alerts and SMS Notification Timing

The system sends an SMS when temperature or BPM readings cross predefined thresholds. An important feature is the timing control, ensuring SMS alerts are sent no more than once every 30 minutes, regardless of sensor reading fluctuations.

```

1 if (tempC > temp_upper_threshold || tempC < temp_lower_threshold || (bpm >
2   bpm_upper_threshold && bpm > 0) || (bpm < bpm_lower_threshold && bpm > 0))
3 {
4   unsigned long currentMillis = millis(); // Get current time in
5   milliseconds
6   // Check if at least 30 minutes have passed since the last SMS
7   if (currentMillis - lastSMSTime >= 1800000) {
8     char smsString[240];
9     sprintf(smsString, "%s's Health Alert: Body Temperature has crossed
10    %.2f C and current BPM is %d.", name, tempC, bpm);
11    if (sendSMS(smsString)) {
12      lastSMSTime = currentMillis; // Update the last SMS timestamp
13      Serial.println("SMS sent successfully");
14    } else {

```

```

11         Serial.println("Failed to send SMS");
12     }
13 }
14 }

```

Listing 6.6: Conditional SMS Notifications Based on Thresholds

These segments of code highlight how the Twilio API is utilized for real-time health monitoring and alerting via SMS, enhancing the Smart Patch's capability to respond to critical health events efficiently.

6.3 Middleware

The Middleware component of the VitalMonitor system plays a crucial role as the intermediary layer that facilitates data communication and integration between the Smart Patch, the Dashboard, and the cloud-based database. Developed using the Flask framework, this Middleware is pivotal in managing the data flow within the system, ensuring that interactions are seamless and efficient.

6.3.1 Connection with Google Cloud

Key configurations of the middleware included establishing a connection to a Google Cloud MySQL instance via Unix sockets, ensuring secure and reliable database interactions. The use of environment variables (.env file) for sensitive information like database credentials was implemented.

```

1 def connect_unix_socket() -> sqlalchemy.engine.base.Engine:
2     """Initializes a Unix socket connection pool for a Cloud SQL instance of
   MySQL."""
3
4     db_user = os.environ["DB_USER"] # e.g. 'my-database-user'
5     db_pass = os.environ["DB_PASS"] # e.g. 'my-database-password'
6     db_name = os.environ["DB_NAME"] # e.g. 'my-database'
7     unix_socket_path = os.environ[
8         "INSTANCE_UNIX_SOCKET"
9     ] # e.g. '/cloudsql/project:region:instance'
10
11     pool = sqlalchemy.create_engine(
12         # Equivalent URL:
13         # mysql+pymysql://<db_user>:<db_pass>@/<db_name>?unix_socket=<
   socket_path>/<cloud_sql_instance_name>
14         sqlalchemy.engine.url.URL.create(
15             drivename="mysql+pymysql",
16             username=db_user,
17             password=db_pass,
18             database=db_name,
19             host='34.147.167.209',
20             query={
21                 "ssl_ca": "./permissions/server-ca.pem",

```

```

22         "ssl_cert": "./permissions/client-cert.pem",
23         "ssl_key": "./permissions/client-key.pem",
24         "ssl_verify_cert": "false"
25     },
26 ),
27 )
28
29 return pool

```

Listing 6.7: Connection to Google Cloud MySQL using unix socket label

6.3.2 Database Integration

The Middleware's database schema was designed to support complex data structures, involving multiple tables such as *Organizations*, *Users*, *Devices*, and *DeviceData*, each serving distinct data storage and retrieval functionalities. SQLAlchemy ORM was utilized for database operations, simplifying CRUD operations and enhancing the maintainability of the code. Flask-Migrate was integrated for handling database migrations, which assists in evolving the database schema over time without losing data.

Security and Performance

Security in data handling is paramount, especially given the sensitive nature of the stored health data. The *Organization* model employs the Flask-Bcrypt library for secure password handling:

```

1 from flask_bcrypt import Bcrypt
2
3 bcrypt = Bcrypt()
4
5 class Organization(db.Model):
6     __tablename__ = 'organizations'
7     password_hash = db.Column(db.String(255), nullable=False)
8
9     def set_password(self, password):
10         self.password_hash = bcrypt.generate_password_hash(password).decode('
11         utf-8')
12
13     def check_password(self, password):
14         return bcrypt.check_password_hash(self.password_hash, password)

```

Listing 6.8: Secure Password Handling in Organization Model

The connection between the middleware and Google Cloud is more secure by using permission files which can only be generated once from the cloud. Without these permissions files, middleware won't get connected to the cloud.

6.3.3 API Integration

API endpoints were carefully crafted to support essential operations such as user and device registration, login processes, and data transmission between devices and the Dashboard. These endpoints handle data validation, session management, and error responses effectively, ensuring a seamless user experience and robust data handling.

User and Organization Management

The API endpoints for registering organizations and users are fundamental for setting up and securing access to the system. They ensure that only authorized entities can register and manage devices and users. Below are examples of these key functionalities:

```

1 @app.route('/register', methods=['POST'])
2 def register_organization():
3     data = request.json
4     name = data.get('name')
5     password = data.get('password')
6     if not name or not password:
7         return jsonify({"error": "Name and password are required"}), 400
8     if Organization.query.filter_by(name=name).first():
9         return jsonify({'error': 'Organization already exists'}), 400
10    new_organization = Organization(name=name)
11    new_organization.set_password(password)
12    db.session.add(new_organization)
13    try:
14        db.session.commit()
15    except Exception as e:
16        db.session.rollback()
17        return jsonify({"error": str(e)}), 500
18    return jsonify({"success": "Organization registered successfully", "organization_id": new_organization.id, "organization_name": name}), 201

```

Listing 6.9: API Endpoint for Organization Registration

This endpoint performs critical checks such as validation of input fields and checks against existing records, ensuring data integrity and preventing duplicates in the system.

Device Management

Device management is crucial for tracking and updating the status and configuration of devices connected to the system. The API allows for device addition, updates, and assignment to users or organizations, as shown in the following example:

```

1 @app.route('/add-new-device', methods=['POST'])
2 def add_or_update_device():
3     data = request.json
4     mac_address = data.get('mac_address')
5     ip_address = data.get('ip_address')
6     passkey = data.get('passkey')

```

```

7     nickname = data.get('nickname')
8     device = Device.query.filter_by(mac_address=mac_address).first()
9     if device:
10         device.ip_address = ip_address
11     else:
12         device = Device(mac_address=mac_address, ip_address=ip_address,
13                         nickname=nickname, passkey=passkey)
14         db.session.add(device)
15     try:
16         db.session.commit()
17         return jsonify({"success": "Device added or updated successfully", "
18                         device_id": device.id}), 201
19     except Exception as e:
20         db.session.rollback()
21         return jsonify({"error": str(e)}), 500

```

Listing 6.10: API Endpoint for Adding or Updating a Device

This endpoint not only facilitates the registration of new devices but also handles updates to existing ones, demonstrating the dynamic nature of device management within the system.

Real-Time Data Interaction

For real-time interaction, APIs facilitate the immediate relay of data between the devices and the system. This includes sending updated configuration details to the devices or receiving real-time data for display on the dashboard.

```

1 @app.route('/get-device-data/<int:device_id>', methods=['GET'])
2 def get_device_data(device_id):
3     def generate():
4         with app.app_context():
5             while True:
6                 latest_data = DeviceData.query.filter_by(device_id=device_id)
7                 .order_by(DeviceData.timestamp.desc()).first()
8                 yield f"data:{json.dumps({'bpm': latest_data.bpm, '
9                     temperature': latest_data.temperature})}\n\n"
10                time.sleep(5) # Adjust as necessary for your application's
11                               needs
12     return Response(generate(), content_type='text/event-stream', headers={'
13                     Cache-Control': 'no-cache'})

```

Listing 6.11: API Endpoint for Real-Time Data Streaming

This streaming API is essential for maintaining a live dashboard that reflects the most current data from connected devices.

These API integrations are pivotal in ensuring that the Middleware efficiently handles all communications within the VitalMonitor system, thus supporting robust, scalable, and secure operations.

6.4 Web Dashboard

The implementation phase of the Web Dashboard for the VitalMonitor system was guided by a commitment to reliability, security, and enhancing user efficiency. Leveraging a streamlined technology stack composed of Flask, HTML, CSS, and JavaScript, the development went beyond merely meeting functional requirements. It also focused on delivering an engaging and effective user experience for organization administrators. Most of the development is very straightforward and didn't involve much complexities unlike for Smart Patch and Middleware. Some important implementations of certain components of the website are discussed below.

6.4.1 Session Management using custom decorators

As per the design discussed in the last chapter, Custom Decorators were used for session management. Custom Decorators are nothing but a function that adds functionality to another function or method without modifying its structure. This is particularly useful in web development for handling tasks like authentication and authorization. In this case provided in Listing 6.12, *@login_required* decorator, it acts as a gatekeeper for routes that require user authentication.

When applied, such as before the *add_device_page()* function, the *login_required* decorator checks whether the *organization_id* exists in the session. This *organization_id* is crucial as it signifies that the admin representing an organization is currently logged in. If the check fails (i.e., if *organization_id* is not found), it redirects the admin to the login page. This prevents unauthorized users or admins from accessing routes that are meant to be secured, thereby protecting sensitive actions like adding devices.

```

1 def login_required(f):
2     @wraps(f)
3     def decorated_function(*args, **kwargs):
4         if 'organization_id' not in session:
5             return redirect(url_for('register_login_page'))
6         return f(*args, **kwargs)
7     return decorated_function
8
9 @app.route('/add-device-page')
10 @login_required
11 def add_device_page():
12     organization_name = session.get('organization_name')
13     return render_template('add_device.html', organization_name=
        organization_name)

```

Listing 6.12: Custom Decorator - login_required

6.4.2 Real-time Data

The real-time data fetching mechanism is used for user pages. Figure 6.2 demonstrated in the JavaScript code utilizes Server-Sent Events (SSE) to establish a continuous stream from

a server to the web browser. Specifically, the function `startLiveDataStream(deviceId)` initiates the connection using the EventSource API, which connects to a specified URL that corresponds to the device ID. Once connected, the server streams data in real-time, sending updates that are automatically received by the client-side JavaScript. Additionally, the function implements robust error handling and auto-reconnect features. If the connection fails or the page becomes invisible, the connection is closed, and attempts to reconnect are made automatically, ensuring continuous data delivery without manual refresh.

```

24 <script>
25 // Function to start listening for live data updates using Server-Sent Events
26 function startLiveDataStream(deviceId) {
27 >   if (!deviceId) { ...
31
32   if (typeof(EventSource) !== "undefined") {
33     let source = new EventSource(`http://localhost:5500/get-device-data/${deviceId}`);
34
35     source.onmessage = function(event) {
36       const data = JSON.parse(event.data);
37       document.getElementById("bpm").textContent = "BPM: " + data.bpm;
38       document.getElementById("temperature").textContent = "Temperature: " + data.temperature + " °C";
39
40       // Threshold checks and visual alerts
41       checkAndUpdateAlerts(data.bpm, data.temperature);
42     };
43
44 >   function checkAndUpdateAlerts(bpm, temperature) { ...
57
58 >   source.onerror = function(event) { ...
64
65   // Close the SSE connection when the page is not visible
66   document.addEventListener('visibilitychange', function() {
67     if (document.visibilityState === 'hidden') {
68       source.close();
69       console.log("SSE closed due to page being hidden.");
70     } else if (!source || source.readyState === EventSource.CLOSED) {
71       startLiveDataStream(deviceId);
72       console.log("Reconnecting SSE as page is visible again.");
73     }
74   });
75
76   } else {
77     document.getElementById("live-data").innerHTML = "Your browser does not support Server-Sent Events.";
78   }
79 }
80
81 </script>

```

Figure 6.2: Fetching User Health Data in real-time.

6.4.3 User Interface and Interaction

The design of the dashboard prioritizes ease of use and a minimalistic approach as seen in Figures 6.3 - 6.8. Two major things that were taken care of while implementing the UI were:

- **User Navigation:** The dashboard layout emphasizes navigational clarity, allowing admins to smoothly transition between viewing all users, managing individual profiles, and device settings without complex navigation steps.
- **Actionable Insights:** Interactive elements such as buttons and links are strategically

placed to facilitate the easy completion of administrative tasks, such as adding or updating user information, managing devices, and monitoring health vitals.

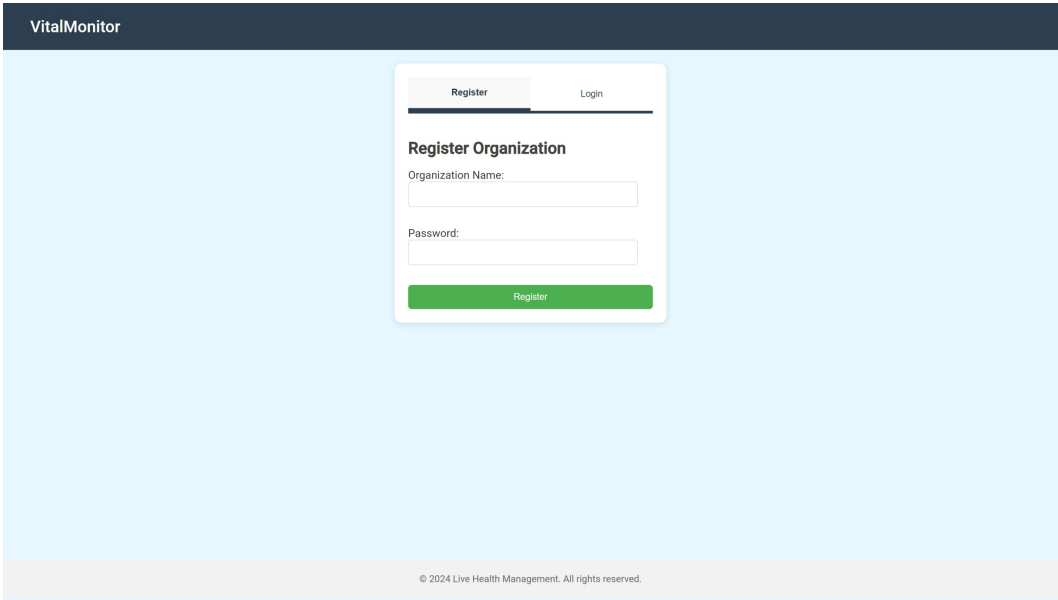


Figure 6.3: Login and Register Page (landing page)

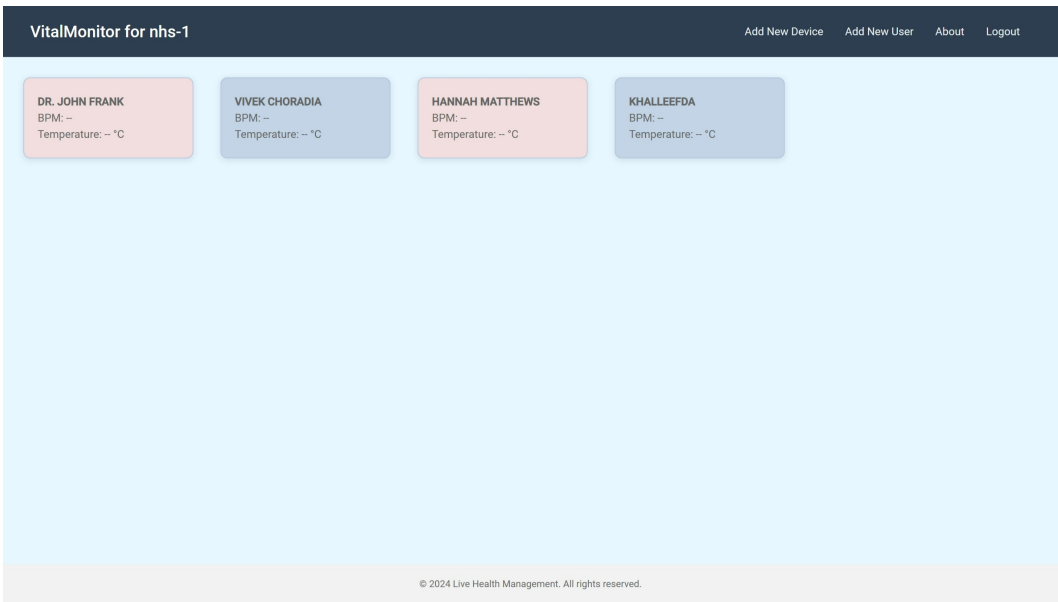


Figure 6.4: Home Page

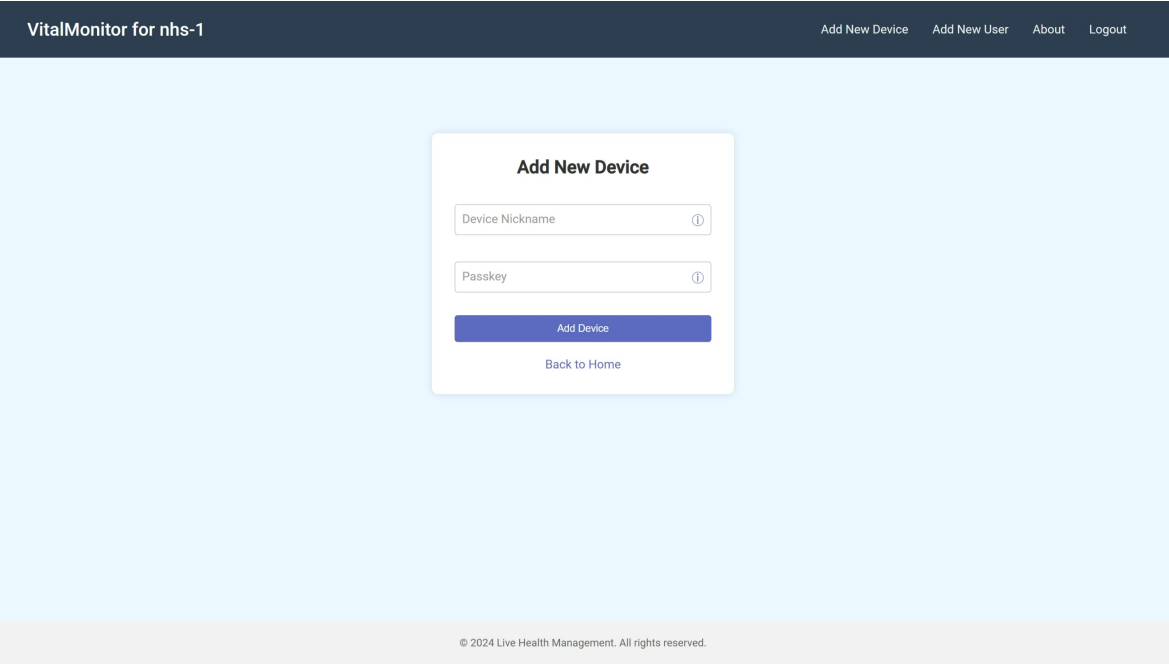


Figure 6.5: Add New Device Page

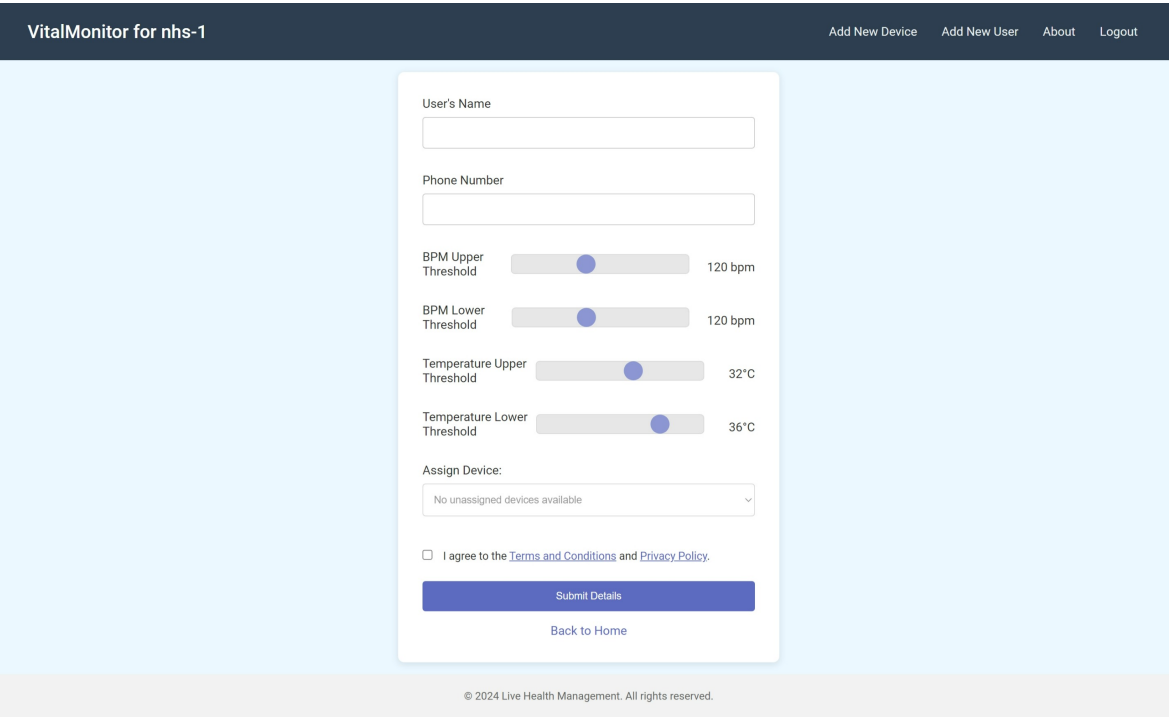


Figure 6.6: Add New User Page

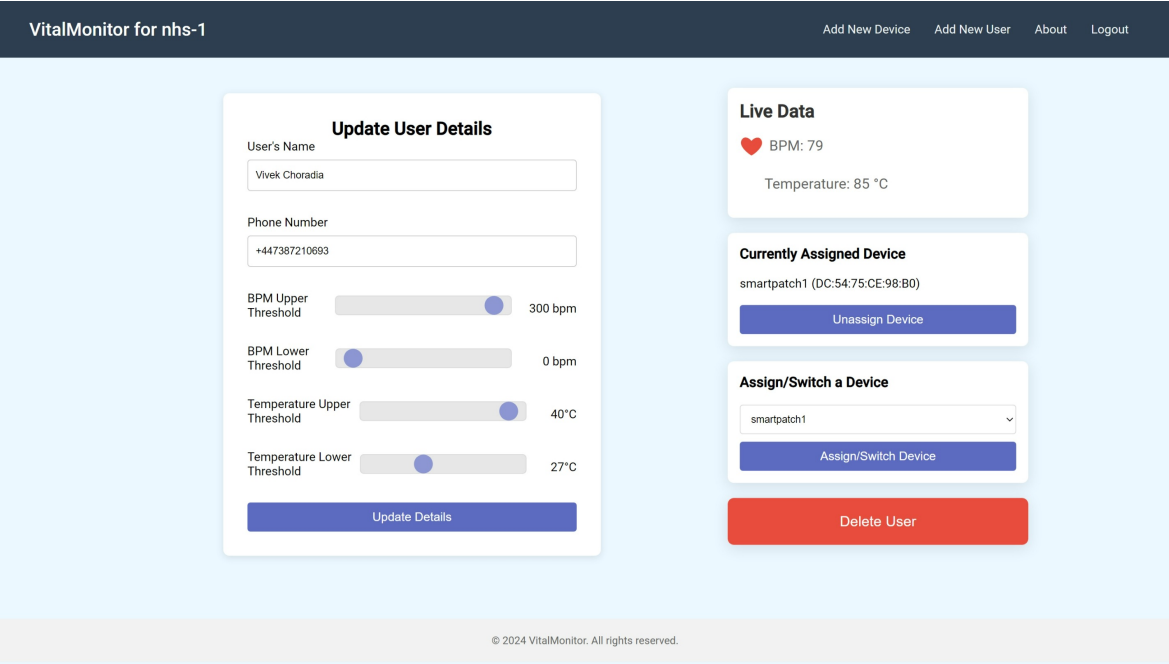


Figure 6.7: User Page

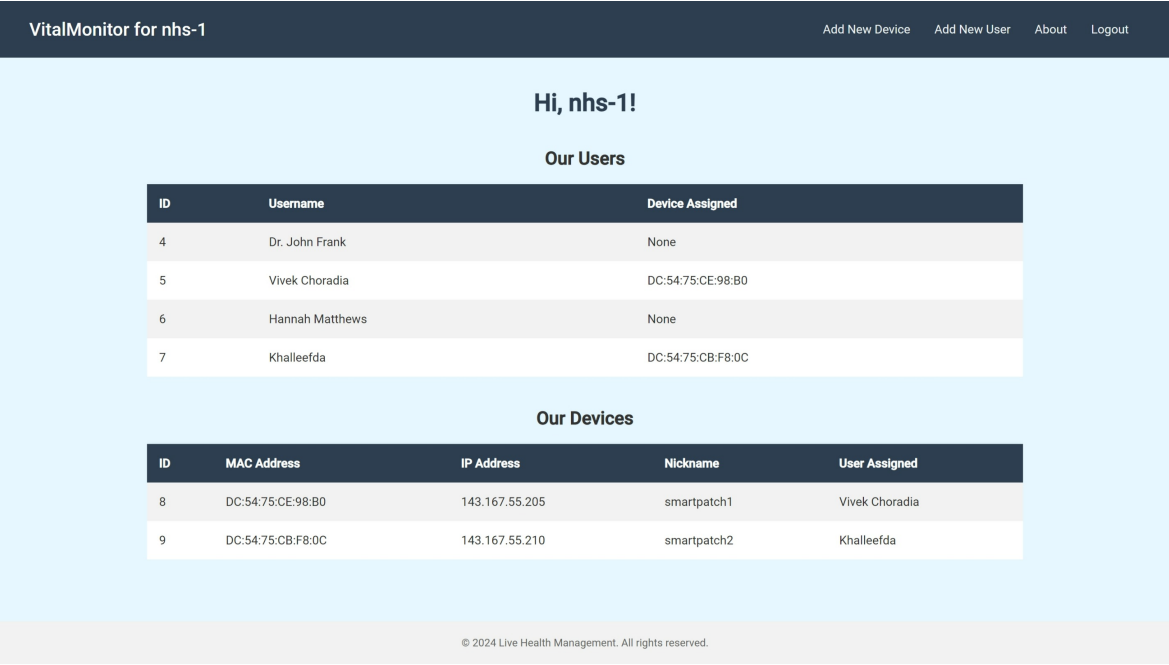


Figure 6.8: About Page

Lastly, Privacy policy (refer to Appendix E) and Terms and Conditions (refer to Appendix F) documents were made. Admin using this dashboard, would have to agree to these documents every time they add a new user. This would force admins to ask for permissions of the user using the device that they agree with how they their data is used and they know their rights.

6.5 Testing

The Testing section details the comprehensive strategies and methodologies used to validate the functionality and reliability of VitalMonitor. This encompasses a range of testing techniques from automated unit and integration tests to manual testing, ensuring the system meets all specified requirements.

6.5.1 Acceptance Testing

Acceptance Testing is important in verifying the system against defined user stories and ensuring that the VitalMonitor functions as intended for end-users. Each user story is rigorously tested with clear criteria that determine whether the feature is implemented satisfactorily. These tests can be found in the Table 6.2 below.

Table 6.2: User Stories for VitalMonitor Dashboard Implementation

User Role	Story	Acceptance Criteria	Status
Admin	I want to register my organization.	1. Admin fills and submits a form with organization details. 2. System confirms registration.	Implemented
Admin	I want to log in to the dashboard.	1. Admin enters credentials. 2. System authenticates and grants access. 3. Error messages are displayed for failures.	Implemented
Admin	I want to add new users.	1. Admin fills out user form including essential details. 2. Optionally assigns a device. 3. Privacy policy agreement is required for submission.	Implemented
Continued on next page			

Table 6.2 Continued from previous page

User Role	Story	Acceptance Criteria	Status
Admin	I want to add new devices.	1. Admin enters device details. 2. System validates and registers the device. 3. Admin receives confirmation.	Implemented
Admin	I want to manage user-device assignments.	1. Admin can assign, unassign, or switch devices through user profiles. 2. Updates are immediate and visible.	Implemented
Admin	I want to monitor vital signs in real-time.	1. Live vital data displayed on user cards. 2. Detailed view available upon clicking a card. 3. Data streams live.	Implemented
Admin	I want to securely log out.	1. Admin clicks logout in Navbar. 2. Redirected to login page. 3. Session is cleared.	Implemented
Admin	I want to view a summary of users and devices.	1. Admin accesses About Page from Navbar. 2. Page lists all resources. 3. Options for data reporting and exporting available.	Implemented
User	I want to wear the device comfortably on my ear (Future Implementation).	1. The device is lightweight and designed to fit securely on the ear. 2. Users report comfort and lack of interference with daily activities.	Not Started
User	I want to easily activate the Smart Patch by pressing a button.	1. The device has a single button for activation. 2. Pressing the button turns on the device and starts data transmission, indicated by a red light.	Implemented
Continued on next page			

Table 6.2 Continued from previous page

User Role	Story	Acceptance Criteria	Status
User	I want to press the button again to stop sending data.	1. Pressing the button a second time turns off the device and stops data transmission, indicated by the red light turning off. 2. Data stops being published immediately.	Implemented
User	I want the device to continuously measure my heart rate and body temperature while activated.	1. The device measures heart rate and body temperature in real-time. 2. Data is sent to the system at regular intervals without delay while the device is on.	Implemented

6.5.2 System and Integration Testing

System and Integration Testing play a crucial role in ensuring the reliability and functionality of VitalMonitor's API endpoints. These tests aim to validate the behavior of the system as a whole and the interactions between its various components. VitalMonitor employs a testing approach, primarily using curl commands to interact with API endpoints. This approach allows for thorough validation of each endpoint's functionality and behavior in different scenarios.

Various testing scenarios are covered to ensure the robustness of VitalMonitor's API endpoints:

1. **Endpoint Functionality:** Each API endpoint is tested to verify its functionality, including creating, updating, retrieving, and deleting resources such as organizations, users, and devices.
2. **Data Validation:** Input data is tested for validity and completeness, ensuring that endpoints handle both valid and invalid data gracefully. This includes testing edge cases and boundary conditions.
3. **Error Handling:** Error scenarios are tested to validate the system's behavior when encountering errors, such as missing parameters, unauthorized access, or internal server errors. The responses are examined to ensure they provide meaningful error messages and appropriate status codes.
4. **Integration Testing:** Endpoints that interact with external systems, such as devices

or external APIs, are thoroughly tested to ensure seamless integration and proper data exchange.

Curl commands are utilized to perform these testings of VitalMonitor's API endpoints. These commands are executed from the command line interface and simulate HTTP requests to interact with the system. One example of such command and the response received is shown in Figure 6.9:

```
C:\Users\vivek>curl -X POST -H "Content-Type: application/json" -d "{\"name\":\"example_org\", \"password\":\"example_password\"}" http://localhost:5500/register
{
  "organization_id": 8,
  "organization_name": "example_org",
  "success": "Organization registered successfully"
}
```

Figure 6.9: CURL Command to test whether adding organization

6.5.3 Unit Testing

Unit testing for both the middleware and the web dashboard was rigorously conducted to ensure robustness and functionality across all components. These tests were implemented using the *pytest* framework and the *flask-testing* library, supplemented by additional tools such as *contextlib*, *unittest.patch*, *unittest.mock*, *sqlalchemy.exc*, and *requests.exceptions*. This robust toolkit facilitated a comprehensive testing approach, encompassing both functional correctness and error handling capabilities.

To avoid impacting the development database, a dedicated local SQLite database was provisioned for the duration of the testing phase. This temporary database was programmatically created prior to each test suite and subsequently destroyed upon completion, ensuring a clean slate for each test run and preventing data persistence that could skew test outcomes.

Mock responses and requests were meticulously crafted to simulate API interactions without live network calls, allowing controlled error testing. For cases where API interactions could not be effectively simulated, manual testing provided comprehensive coverage.

Coverage analysis was conducted using the *coverage.py* tool, providing insights into the extent of code executed by the tests and highlighting any areas lacking in test coverage. This analysis was crucial in identifying untested paths and informed subsequent test development to ensure a high degree of code coverage.

The results of these efforts are as follows:

- The middleware was subjected to 37 unit tests, with the coverage outcomes detailed in Figure 6.10.

Coverage report: 82%

coverage.py v7.4.4, created at 2024-04-21 21:31 +0100

Module	statements	missing	excluded	coverage
app__init__.py	26	4	0	85%
app\cloudsql.py	9	0	0	100%
app\config.py	10	0	0	100%
app\models.py	45	0	0	100%
app\routes.py	276	105	0	62%
tests__init__.py	0	0	0	100%
tests\conftest.py	16	0	0	100%
tests\unit__init__.py	0	0	0	100%
tests\unit\test_models.py	60	0	0	100%
tests\unit\test_routes.py	186	6	0	97%
tests\unit\test_sql.py	24	2	0	92%
Total	652	117	0	82%

coverage.py v7.4.4, created at 2024-04-21 21:31 +0100

Figure 6.10: Middleware Coverage

- The web dashboard underwent a series of 27 unit tests, with its respective coverage report depicted in Figure 6.11.

Coverage report: 88%

coverage.py v7.4.4, created at 2024-04-21 21:42 +0100

Module	statements	missing	excluded	coverage
app__init__.py	13	0	0	100%
app\routes.py	189	26	0	86%
config.py	13	0	0	100%
tests__init__.py	0	0	0	100%
tests\unit__init__.py	0	0	0	100%
tests\unit\test_routes.py	233	27	0	88%
Total	448	53	0	88%

coverage.py v7.4.4, created at 2024-04-21 21:42 +0100

Figure 6.11: Web Dashboard Coverage

6.5.4 Manual Testing

Manual testing was rigorously conducted on both the Web Dashboard and the Smart Patch to ensure comprehensive validation of system functionalities not covered by automated tests. This process is crucial for verifying the integrated operation of all subsystems, offering insights into real-world usability and interaction sequences that automated tests might overlook.

For a structured approach, each test case was assigned a unique Test Case ID, with meticulously detailed test procedures and clearly defined expected results. The outcomes of these tests were diligently recorded, specifying whether each test case passed or failed, thus providing a transparent and accountable testing framework.

The example below illustrates the methodical nature of our manual testing approach:

Table 6.3: Manual Test Cases for Smart Patch Device

Test Case ID	Test Details	Expected Results	Pass/Fail
TC-SP01	Verify activation of the Smart Patch by pressing the button. - Ensure the device is off. - Press the activation button.	Device turns on (red light indicator) and starts transmitting data.	Pass

A comprehensive list of all manual test cases, along with their respective details and outcomes, is provided in the **Appendix G**. This meticulous documentation facilitates an in-depth understanding of the product’s performance in real-world scenarios and ensures all subsystems function harmoniously together.

Chapter 7

Results and Discussions

This chapter presents a comprehensive analysis of the results obtained from the development and implementation of the VitalMonitor. The primary objective was to address existing gaps in healthcare monitoring of HCPs by using IoT and IoTaaS platform to facilitate remote and non-intrusive healthcare solution for HCPs. The findings are evaluated against the project's original aims outlined in Chapter Three, Requirements and Analysis. The final product's demonstration video can be seen **here**. This google drive link is only accessible by University of Sheffield email address.

7.1 Revisited Requirements

This project successfully met the majority of its objectives. A prototype of Smart Patch was created. However, the creation of a fully realized product was impeded by several constraints. The prototype is able to get heart rate and body temperature and transmit it in real-time to the dashboard. Additionally, it features functionality to send alerts to emergency contacts via the Twilio API. While the prototype demonstrates substantial capabilities, the transition to a full-scale product would necessitate expertise in electrical engineering and proficiency in CAD for 3D printing.

Aside from the Smart Patch, the IT infrastructure and web dashboard of VitalMonitor has been fully implemented and is operational. The web dashboard is designed specifically for organizational administrators, offers comprehensive functionalities including the ability to add and assign devices to users, among other tasks. These features fulfill all stated project requirements.

During the project's lifecycle, several requirements were revisited and refined to better align with the evolving technological landscape and user feedback. This adaptive approach was pivotal in addressing unforeseen challenges and leveraging new opportunities as they arose. A notable adjustment was the substitution of the LM35 temperature sensor with the DS18B20 sensor. This change was motivated by the digital output capabilities of the DS18B20, which

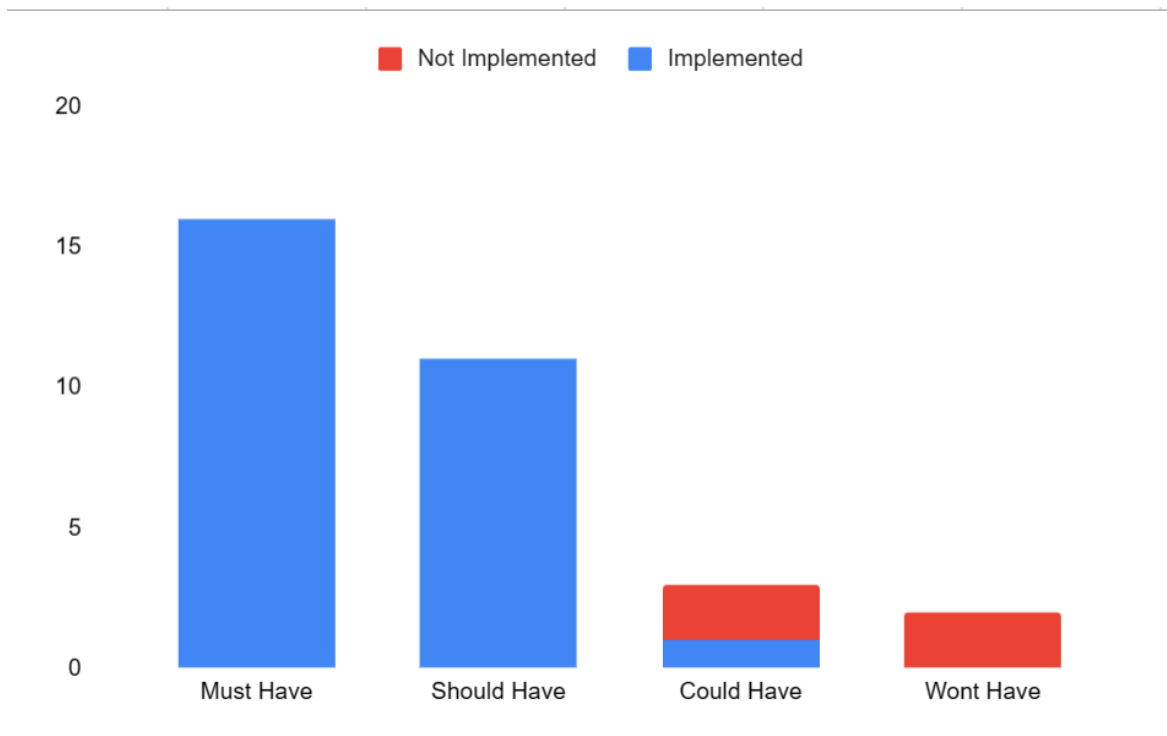


Figure 7.1: Overview of Requirements post development

facilitate easier data integration and enhance the accuracy of measurements.

7.2 Project Cost

The funding required to design, implement, and test the entire project was substantial and supported by the Department of Computer Science. Monitoring the expenses associated with the prototype provides insights into potential costs of a fully developed product. Minimizing these costs is critical for promoting adoption by healthcare organizations, potentially alleviating their financial constraints.

A detailed overview of the project's expenditures is provided in Table 7.1 to ensure transparency in the allocation and utilization of resources.

Additional Costs:

- Utilization of \$55 from Google Cloud's \$300 free credits for development purposes. Cost per day to run the database on Google Cloud with current configurations are \$0.4/day.
- Employment of the AdafruitIO free tier, which includes access to 2 dashboards and 10 feeds — sufficient for this project phase.

Item	Cost (£)
Pimoroni Pulse Sensor x 2	42
LM-35 Sensor	6.24
ESP32-S3-MINI-1	21.00
DS18B20	3.50
Miscellaneous Components	4.59
Total Hardware Cost	56.33

Table 7.1: Project Cost

- Allocation of \$12 from a Twilio trial credit to purchase a UK phone number, with each text message incurring a cost of \$0.0420.

7.3 VitalMonitor’s IT Infrastructure

VitalMonitor has been designed and implemented with scalability and versatility as foundational principles. The IoTaaS platform enables registration for multiple organizations, each of which can support numerous users. VitalMonitor is engineered to easily incorporate additional sensors and functionalities, enhancing its capacity to monitor parameters indicative of fatigue or stress levels.

A significant development was the introduction of middleware, which expanded the adaptability of VitalMonitor. Originally conceived to operate within a single hospital’s local network, the middleware now facilitates the connection of Smart Patches across diverse network environments. In future iterations, incorporating a System on Chip (SoC) with an integrated SIM card could enable autonomous network connectivity, allowing data to be sent directly to middleware hosted on cloud services such as Google Cloud. This feature would allow organizational administrators to access the dashboard from any location, greatly enhancing the system’s utility.

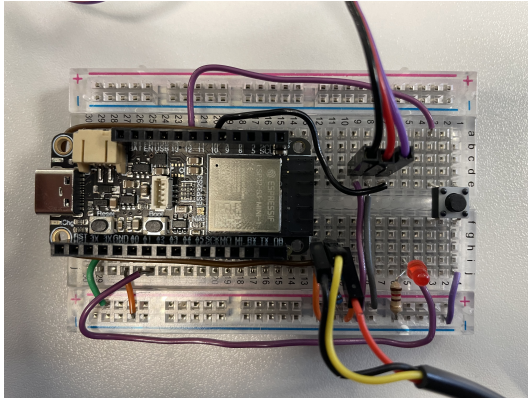
Such flexibility extends the potential applications of the product to include healthcare providers in ambulances, natural disaster response teams, and medical camps in remote areas. This adaptability underscores the advanced capability of VitalMonitor to meet a wide range of healthcare monitoring needs, making it a highly versatile and indispensable tool in diverse medical contexts.

7.4 Prototype of Smart Patch

Smart Patch was implemented keeping in mind its primary objective of being the device that doesn’t hinder HCPs in their everyday work. Smart Patch’s prototype was made in this iteration of the project. The prototype was specifically designed and developed using sensors which serve this theme. It is able to detect vitals of HCP in the least intrusive and

most remote way. It runs its own mini asynchronous web server to transmit the data to middleware server. Additionally, a manual switch is incorporated, empowering the wearer to control when data is recorded. Only when the wearer (here HCP) presses the switch and red LED turns on - only then the data is measured and published.

The ergonomic design of the device, strategically positioned behind the ear, and the selection of sensors were dictated by the need for comfort and the physiological requisites for precise data collection. This stage of the project did not focus on finalizing the product design; thus, further development involving 3D printing and product optimization is required. Time constraints in this phase prevented the advancement to a finalized product.



(a) Prototype



Generated using Dall-E

(b) Design Idea of Physical Product

Figure 7.2: Smart Patch

7.5 Limitations and Issues Encountered

7.5.1 Trade off between Sensor Accuracy and Non-Intrusiveness

In developing wearable devices for continuous monitoring, a significant challenge is balancing sensor accuracy with the need for non-intrusiveness, particularly for healthcare professionals (HCPs) who wear these devices during long shifts. High accuracy in sensors is crucial, as it ensures that the data collected is reliable enough to inform meaningful interventions to support HCP well-being. However, achieving this often requires more invasive technologies that may interfere with the daily activities and comfort of these professionals.

VitalMonitor focuses on optimizing this balance by selecting sensors that minimize intrusiveness while still providing sufficiently accurate data for passive monitoring. This approach is critical in settings where both the well-being of HCPs and operational efficiency are prioritized. Given the limited availability of sensors that strike a perfect balance between non-

intrusiveness and high accuracy, the selection was guided by the goal of maintaining user comfort and system reliability. This ensures that healthcare administrators can effectively monitor and respond to the health indicators of their staff without causing discomfort or disruption to their work.

7.5.2 Settings Get Lost When Power Goes Off

The Smart Patch operates by assigning itself to a user through the dashboard. Once the assignment is complete, user data is received by the device. Publishing data only starts when the switch is pressed on the device. However, once the Smart Patch loses power, all assigned data is lost.

To mitigate this issue, the flash memory available in the ESP32 is utilized. EEPROM (Electrically Erasable Programmable Read-Only Memory) is used to save essential data, so the admin does not need to reassign the Smart Patch to a user after power loss. EEPROM is a non-volatile memory technology that retains important data unless the user is deleted or the device is unassigned or switched.

When the device powers on, it first checks for saved information or settings. If data already exists, it will resume publishing data as previously configured. Here is a code snippet illustrating how EEPROM is used in the setup process:

```

1  if (!EEPROM.begin(512)) {
2      Serial.println("Failed to initialize EEPROM");
3      return;
4  }
5
6  DeviceSettings settings = loadSettings();
7  global_name = settings.name;
8  global_to = settings.phoneNumber;
9  global_bpm_lower_threshold = settings.bpmLowerThreshold;
10 global_bpm_upper_threshold = settings.bpmUpperThreshold;
11 global_temp_lower_threshold = settings.tempLowerThreshold;
12 global_temp_upper_threshold = settings.tempUpperThreshold;
13 global_isPublishing = settings.isPublishing;

```

Listing 7.1: Using EEPROM in setup()

This approach ensures that critical settings stay in the Smart Patch unless explicitly changed.

7.5.3 Converting Raw Data to Useful Data

The DS18B20 Temperature Sensor and Pimoroni Pulse Sensor are integral to VitalMonitor's functionality, changing analog electrical signals into digital data for health monitoring. However, this reading process often introduces significant noise, resulting in irregular and unreliable data outputs. Such anomalies, if unaddressed, could potentially trigger unwarranted

alarms or actions based on incorrect information. This necessitates robust data processing strategies to ensure accuracy and usability.

A common challenge in sensor data analysis is managing these irregularities, particularly when they lead to significant deviations from expected patterns. For example, an abrupt spike in temperature readings from 32 degrees Celsius to an erroneous 45 degrees could inadvertently exceed the system's threshold of 35 degrees, resulting in false alarms. To ensure data integrity and reliability, it is imperative to employ robust data smoothing techniques.

The Moving Average filter is one such technique applied within the VitalMonitor framework. By averaging data points over a defined time window, this method effectively dampens the impact of transient spikes or noise, leading to a more stable and accurate data output. This technique not only enhances the overall quality of the health monitoring data but also supports critical decision-making processes by reducing the likelihood of false alerts. Figure 7.3 provides a snapshot of data collected from the sensors in their raw form as well as data after implementing Moving Average function to it.

Extensive testing and evaluation have confirmed that the adapted sensors provide a data precision of ± 15 BPM for heart rate and $\pm 0.2 - 0.5$ degree Celsius for temperature. These measures ensure that VitalMonitor delivers reliable and actionable health data for healthcare professionals, enhancing the monitoring and management of their well-being.

7.5.4 Reliability depends on Network Bandwidth

The effective operation of VitalMonitor's Smart Patches heavily relies on the strength and capacity of the WiFi network to which they are connected. These devices are designed to upload data in real-time, a process that demands robust network performance to function seamlessly. With a scenario involving up to 100 Smart Patches transmitting data simultaneously, the bandwidth requirements become substantial. This substantial demand can strain the network, leading to potential data transmission delays that may impact the timeliness and reliability of data being displayed on the monitoring dashboard.

During extensive testing and evaluation, it was observed that the system's reliability is significantly influenced by the WiFi network's bandwidth. Insufficient bandwidth can result in lagging data updates, which may hinder the system's ability to provide real-time insights crucial for immediate decision-making and response. This finding underscores the necessity for implementing a network infrastructure capable of supporting high data loads, especially in environments where continuous and simultaneous monitoring is critical. Efforts to enhance network configurations, possibly through dedicated channels or enhanced WiFi protocols, are essential to mitigate these issues and ensure reliable system performance.

Time	Raw Data	Moving Average Data
14:36:02.866	BPM: 20	BPM: 20
14:36:03.667	Temperature: 25.75	Temperature: 25.75
14:36:05.875	BPM: 48	BPM: 48
14:36:06.648	Temperature: 25.75	Temperature: 25.75
14:36:08.895	BPM: 52	BPM: 52
14:36:09.667	Temperature: 25.75	Temperature: 25.75
14:36:11.854	BPM: 0	BPM: 52
14:36:12.640	Temperature: 25.81	Temperature: 25.81
14:36:14.863	BPM: 0	BPM: 52
14:36:15.672	Temperature: 25.81	Temperature: 25.81
14:36:17.868	BPM: 144	BPM: 124
14:36:18.630	Temperature: 25.81	Temperature: 25.81
14:36:20.857	BPM: 120	BPM: 120
14:36:21.674	Temperature: 25.81	Temperature: 25.81
14:36:23.886	BPM: 125	BPM: 125
14:36:24.685	Temperature: 25.81	Temperature: 25.81
14:36:26.884	BPM: 115	BPM: 115
14:36:27.653	Temperature: 25.87	Temperature: 25.87
14:36:29.893	BPM: 145	BPM: 130
14:36:30.655	Temperature: 25.87	Temperature: 25.87
14:36:32.866	BPM: 100	BPM: 100
14:36:33.633	Temperature: 25.87	Temperature: 25.87
14:36:35.891	BPM: 0	BPM: 100
14:36:36.645	Temperature: 25.87	Temperature: 25.87
14:36:38.873	BPM: 0	BPM: 100
14:36:39.628	Temperature: -127	Temperature: 25.87
14:36:41.860	BPM: 110	BPM: 110
14:36:42.649	Temperature: 25.87	Temperature: 25.87
14:36:44.870	BPM: 115	BPM: 115
14:36:45.653	Temperature: 27.06	Temperature: 27.06
14:36:47.863	BPM: 113	BPM: 113
14:36:48.634	Temperature: 28.69	Temperature: 28.69

Figure 7.3: Data from sensors

7.5.5 Google Cloud

Google Cloud offers \$300 in credits for database and server usage. In this project iteration, these credits were primarily utilized for the MySQL database, specifically the Enterprise Cloud SQL Edition. The chosen configuration, as depicted in Figure 7.4, was optimized for cost efficiency, with an initial setup cost of \$6 and a daily operational cost of \$0.4. While suitable for development and testing purposes, this configuration may prove inadequate for production deployment, particularly when supporting a larger user base.

Summary

Region	europa-west2 (London)
DB version	MySQL 8.0.31
vCPUs	1 vCPU
Memory	3.75 GB
Data cache	Disabled
Storage	10 GB
Connections	Public IP
Backup	Automated
Availability	Single zone
Point-in-time recovery	Enabled
Network throughput (MB/s) ?	250 of 250
Disk throughput (MB/s) ?	Read: 4.8 of 200.0 Write: 4.8 of 200.0
IOPS ?	Read: 300 of 12,000 Write: 300 of 10,000

Figure 7.4: Google Cloud Configurations

In a production environment scenario accommodating 100 HCPs and a single dashboard for hospital, upgrading database to the Enterprise Plus Cloud SQL Edition becomes imperative. This entails provisioning a minimum of 8 virtual CPUs, 32 GB of RAM, and increasing storage capacity to over 100 GB. Additionally, as the platform scales to serve multiple hospitals, resource requirements will escalate accordingly.

Furthermore, the middleware microservices will be deployed on Google Cloud in the production phase. These services interact with the database through HTTPS requests, utilizing unix-sockets with appropriate permissions. For instance, when an administrator adds a new device via the dashboard, the workflow involves traversing through the web page, web server, middleware, and database (hosted on Google Cloud) before returning. While deploying middleware on Google Cloud can streamline transactions and improve response times, it necessitates additional costs. Hence, this optimization was deferred in the current iteration to maintain cost-effectiveness.

7.6 Further Work

Despite having achieved the core objectives and beyond, there are more enhancements that this project could have done if not for the time constraint and knowledge-specific constraint.

Further Work are divided into 3, things to be done to create a minimum viable product, improving current implementations and finally adding new requirements to make VitalMonitor more efficient.

7.6.1 Smart Patch: Final Product

The culmination of the Smart Patch project will yield a refined, compact device designed to discreetly rest behind the ear. This final iteration will feature optimized sensor placement, with the pulse sensor delicately positioned on the earlobe and the temperature sensor situated inconspicuously behind the ear. Transitioning from the current breadboard prototype to a fully soldered device with a 3D-printed cover will be pivotal, streamlining the product for mass production and enhancing its durability. A sample design was created for future device using Dall-E. This design is shown in Figure 7.2. Refer to Appendix H for more information on how this image was generated.

To achieve this, future efforts will focus on miniaturizing the device through advanced CAD design and employing more efficient microcontroller tailored specifically for the Smart Patch. However, before deployment, rigorous testing, including Electrical Safety Testing and Mechanical/Physical Testing, will be necessary to ensure compliance with industry standards and user safety. Although these tests were not feasible within the project's constraints, they remain critical future steps to validate the device's reliability and regulatory compliance.

7.6.2 Improving the Middleware Infrastructure

In future, the Middleware Infrastructure can be transitioned from a monolithic architecture to a more scalable and robust system comprising four distinct microservices. Currently, the infrastructure operates as a single web server, but the proposed evolution involves isolating functionality into four dedicated microservices: Session Management Service, User and Device Management Service, Data Ingestion Service, and Data Retrieval Service. By segmenting these services across different web servers, the workload is distributed more evenly, reducing the strain on any single server and enhancing system efficiency.

This architectural enhancement not only fosters a clearer separation of concerns but also facilitates improved request management and scalability. Additionally, implementing a load balancer to orchestrate requests among these microservices ensures efficient data flow, minimizing the risk of data loss and optimizing overall system performance. This strategic restructuring lays the foundation for a more resilient and adaptable Middleware Infrastructure, poised to accommodate future growth and evolving user demands.

7.6.3 Algorithm to Measure Stress and Fatigue Levels

Existing methodologies lack a standardized approach to real-time measurement of stress and fatigue, primarily due to their multifaceted nature, influenced by a multitude of physiological

and psychological factors. To address this challenge, further research is essential to develop an algorithm capable of quantifying these parameters quantitatively. This algorithmic approach aims to integrate various vital signs and personal information, providing a more comprehensive understanding of individual well-being beyond subjective descriptors like “very stressed.” Collaboration with medical researchers will be pivotal in discerning relevant parameters and assigning appropriate weights based on empirical evidence. By leveraging insights from medical literature and empirical data, the project will look in to refining decision-making processes and enhance the precision of stress and fatigue assessments.

The ultimate goal is to streamline healthcare interventions and reduce reliance on simplistic thresholds, fostering a more nuanced approach to individual well-being monitoring. Although this entails extensive data gathering and analysis, its potential to expedite decision-making processes and improve healthcare outcomes underscores its significance in the realm of healthcare monitoring for HCPs.

7.6.4 Interdisciplinary Collaboration for Enhanced Sensors

Addressing the inherent trade-off between sensor accuracy and non-intrusiveness, it is necessary to have interdisciplinary collaboration by electrical engineers and bio-medical engineers in this project. By pooling expertise from both fields, development of enhanced sensors will take place. These sensor(s) will be capable of accurately measuring vital signs such as heart rate and body temperature, while remaining non-intrusive to wear. This collaborative effort holds the potential to disrupt the healthcare industry by introducing innovative sensor technologies that prioritize both accuracy and user comfort.

The development of non-intrusive sensors not only meets the demands of HCPs but also enhances the overall experience for patients undergoing medical treatments. By minimizing discomfort associated with traditional monitoring methods, non-intrusive sensors offer a more patient-centric approach to healthcare delivery, ultimately alleviating suffering and improving the quality of care. This interdisciplinary collaboration represents a pivotal step towards realizing the full potential of sensor technology in revolutionizing healthcare monitoring and patient outcomes.

7.6.5 Prediction Model

Stress and fatigue often rise gradually, with occasional spikes triggered by unforeseen events or stressors. Recognizing this pattern, future iterations will aim to develop a prediction model trained on actual human data. By forecasting stress levels before they surpass predetermined thresholds, this model provides early warning signals to healthcare organization administrators. Such anticipatory measures afford administrators more time to make necessary arrangements, mitigating the impact of heightened stress levels on healthcare professionals (HCPs) and optimizing patient care.

The integration of predictive analysis represents a notable advancement, augmenting Vital-Monitor with cutting-edge AI and analytics functionalities. This strategic augmentation not only distinguishes the platform from competitors but also empowers healthcare organizations with foresight capabilities.

Chapter 8

Conclusion

The healthcare industry has witnessed a significant rise in stress levels among healthcare professionals (HCPs), highlighting the urgent need to prioritize their well-being. Research reveals that the emotional and physical burdens of high-stress environments can profoundly affect HCPs' mental and physical health. Despite significant technological advancements, existing healthcare systems lack comprehensive tools tailored specifically to safeguard the health and well-being of HCPs. This pressing issue calls for a solution that addresses the critical gaps in real-time monitoring and proactive intervention. This led the development of VitalMonitor.

VitalMonitor was designed as an innovative system which tackles this problem head-on. At its core is the Smart Patch, one of the first IoT devices specifically engineered to remotely monitor the health of HCPs in a non-intrusive manner. This device measures critical health parameters like heart rate and body temperature, providing real-time insights into stress and fatigue levels. Coupled with an Internet of Things as a Service (IoTaaS) platform, VitalMonitor enables centralized monitoring through a comprehensive web dashboard. The dashboard offers a seamless user experience for healthcare administrators to access health data, generate alerts, and make proactive decisions that help ensure HCPs' safety.

A comprehensive literature review revealed the limitations of current healthcare monitoring systems and identified the unique opportunity to fill this void. Existing solutions are primarily focused on patient care and often overlook the well-being of healthcare professionals themselves. This gap prompted the project to develop Smart Patch and web dashboard using an agile methodology. The iterative development process ensured continuous refinement of both the hardware and software components, with thorough testing at each stage.

The design phase involved carefully choosing the most suitable technologies, ensuring compatibility between the Smart Patch firmware, middleware, and dashboard components. The ESP32-S3 microcontroller was selected for its built-in wireless connectivity and low power consumption. It was paired with Pimoroni Pulse Sensor and DS18B20 temperature sensor

to achieve accurate, non-intrusive monitoring. The middleware used Google Cloud's secure MySQL infrastructure for efficient data management. Finally, the dashboard was built with Python's Flask framework for robust backend support and easy integration.

The Smart Patch's firmware underwent multiple iterations to enhance its accuracy and reliability. The middleware's API endpoints were rigorously tested to ensure secure data transmission between the device and the dashboard. The dashboard's user interface was designed to prioritize ease of use, allowing administrators to seamlessly monitor HCPs' health metrics in real time. The entire system was subject to unit, integration, and system testing, with special attention to user stories that capture essential functionality.

VitalMonitor's approach aligns with global health trends, addressing the increasing demand for IoT-based wearable health technology. The system was developed with practical implications in mind, focusing on seamless integration with existing healthcare workflows. This ensures that healthcare organizations can immediately benefit from the system by easily incorporating it into their daily work.

While the Smart Patch and dashboard excelled in achieving real-time health monitoring, certain limitations were identified. The system is dependent on network availability, which may impact performance in low-bandwidth environments. Achieving non-intrusive sensor accuracy remains a challenge, necessitating the use of different algorithms such as Moving Average. However, these challenges also present opportunities for future enhancement. By refining middleware infrastructure, implementing prediction models for stress and fatigue, and improving sensor technology through interdisciplinary collaboration, the system's reliability and usability can be further improved.

VitalMonitor has the potential to become a leading tool in redefining healthcare monitoring systems. Its proactive approach can significantly impact global healthcare delivery by reducing HCP burnout and ensuring patient safety. The project serves as a testament to the innovation and dedication that went into creating this solution, opening the door to future research and collaboration to improve HCP well-being further. Healthcare organizations, researchers, and developers should consider adopting or building upon VitalMonitor to advance healthcare technology that protects those who protect us all.

Bibliography

- [1] J. M. Givens Jane L. MD; Tjia, “Depressed medical students’ use of mental health services and barriers to use,” *Academic Medicine*, vol. 77, no. 9, pp. 918–921, 2002.
- [2] Y.-Q. Gao, B.-C. Pan, W. Sun, H. Wu, J.-N. Wang, and L. Wang, “Anxiety symptoms among chinese nurses and the associated factors: A cross sectional study,” *BMC Psychiatry*, vol. 12, no. 1, p. 141, 2012. [Online]. Available: <https://doi.org/10.1186/1471-244X-12-141>.
- [3] N. Khamisa, B. Oldenburg, K. Peltzer, and D. Ilic, “Work related stress, burnout, job satisfaction and general health of nurses,” *International Journal of Environmental Research and Public Health*, vol. 12, no. 1, pp. 652–666, 2015, ISSN: 1660-4601. DOI: 10.3390/ijerph120100652. [Online]. Available: <https://www.mdpi.com/1660-4601/12/1/652>.
- [4] M. K. Bidwal, E. J. Ip, B. M. Shah, and M. J. Serino, “Stress, drugs, and alcohol use among health care professional students: A focus on prescription stimulants,” *Journal of Pharmacy Practice*, vol. 28, no. 6, pp. 535–542, 2015, PMID: 25124376. DOI: 10.1177/0897190014544824. eprint: <https://doi.org/10.1177/0897190014544824>. [Online]. Available: <https://doi.org/10.1177/0897190014544824>.
- [5] J. Dudman, A. Isaac, and S. Johnson, “Revealed: How the stress of working in public services is taking its toll on staff,” *The Guardian*, vol. 10, 2015.
- [6] V. Ceri and I. Cicek, “Psychological well-being, depression and stress during covid-19 pandemic in turkey: A comparative study of healthcare professionals and non-healthcare professionals,” *Psychology, Health & Medicine*, vol. 26, no. 1, pp. 85–97, 2021, PMID: 33320723. DOI: 10.1080/13548506.2020.1859566. eprint: <https://doi.org/10.1080/13548506.2020.1859566>. [Online]. Available: <https://doi.org/10.1080/13548506.2020.1859566>.
- [7] L. Udayanga, A. Perera, and L. Dissanayaka, “Covid19induced anxiety, depression and stress among healthcare professionals in sri lanka,” *Asian Pacific Journal of Tropical Medicine*, vol. 15, no. 11, 2022, ISSN: 1995-7645. [Online]. Available: https://journals.lww.com/aptm/fulltext/2022/15110/covid_19_induced_anxiety,_depression_and_stress.3.aspx.

- [8] W. H. Organization, *Stress*, www.who.int, Feb. 2023. [Online]. Available: <https://www.who.int/news-room/questions-and-answers/item/stress>.
- [9] K. O'Connell, *Causes of fatigue and how to manage it*, Healthline, Mar. 2020. [Online]. Available: <https://www.healthline.com/health/fatigue> (visited on 05/03/2024).
- [10] "A shared commitment to quality - nhs england." Accessed on November 21, 2023. (2021), [Online]. Available: <https://www.england.nhs.uk/wp-content/uploads/2021/04/nqb-refreshed-shared-commitment-to-quality.pdf>.
- [11] A. Charles, *Integrated care systems explained*, Accessed on November 11, 2023, 2022. [Online]. Available: <https://www.kingsfund.org.uk/publications/integrated-care-systems-explained>.
- [12] R. Rowbottom. "Uk: Integrated care systems – lessons and outlook from the first six months — simon kucher." Accessed on November 23, 2023. (2023), [Online]. Available: <https://www.simon-kucher.com/en/insights/uk-integrated-care-systems-lessons-and-outlook-first-six-months>.
- [13] Accenture. "The 'new' rules of healthcare provider engagement." Accessed on November 23, 2023. (2021), [Online]. Available: <https://www.accenture.com/us-en/insights/life-sciences/new-rules-healthcare-provider-engagement>.
- [14] "Cipd — learning development strategy and policy — factsheets." Accessed on November 23, 2023. (2022), [Online]. Available: <https://www.cipd.org/uk/knowledge/factsheets/strategy-development-factsheet/>.
- [15] "Cipd — strategic workforce planning." Accessed on November 23, 2023. (2023), [Online]. Available: <https://www.cipd.org/uk/knowledge/guides/strategic-workforce-planning/>.
- [16] C. Cohen, S. Pignata, E. Bezak, M. Tie, and J. Childs, "Workplace interventions to improve well-being and reduce burnout for nurses, physicians and allied healthcare professionals: A systematic review," *BMJ Open*, vol. 13, no. 6, 2023, ISSN: 2044-6055. DOI: 10.1136/bmjopen-2022-071203. eprint: <https://bmjopen.bmj.com/content/13/6/e071203.full.pdf>. [Online]. Available: <https://bmjopen.bmj.com/content/13/6/e071203>.
- [17] "Developing and evaluating workplace health interventions: Employer toolkit." Accessed on November 23, 2023. (), [Online]. Available: <https://www.gov.uk/government/publications/developing-and-evaluating-workplace-health-interventions-employer-toolkit>.
- [18] "Oracle: What is the internet of things (iot)?" Accessed on November 23, 2023. (), [Online]. Available: <https://www.oracle.com/uk/internet-of-things/what-is-iot/#:~:text=What%20is%20IoT%3F>.
- [19] A. Burak, *Iot in healthcare: Top benefits, use cases, and future trends*, Relevant Software, Nov. 2023. [Online]. Available: <https://relevant.software/blog/iot-in-healthcare/> (visited on 05/03/2024).

- [20] “Global connected wearable devices 2019-2022.” Accessed on November 23, 2023. (2023), [Online]. Available: <https://www.statista.com/statistics/487291/global-connected-wearable-devices/#:~:text=Between%202019%20and%202022%2C%20the>.
- [21] *Uniforms and workwear: Guidance for nhs employers - nhs england*. Accessed on November 19, 2023., 2020. [Online]. Available: <https://www.england.nhs.uk/wp-content/uploads/2020/04/Uniforms-and-Workwear-Guidance-2-April-2020.pdf>.
- [22] J. Tee, *Iotaas? clearing the roadblocks to iot-as-a-service adoption — theserverside*, TheServerSide.com, Mar. 2017. [Online]. Available: <https://www.theserverside.com/feature/IoTaaS-The-things-that-are-hindering-IoT-as-a-Service-adoption> (visited on 05/03/2024).
- [23] *Eai iotaas 2024 – 10th eai international conference as iot as a service*, IoTaaS. [Online]. Available: <https://iotaas.eai-conferences.org/2024/> (visited on 05/03/2024).
- [24] T.-S. Lin, P.-Y. Liu, and C.-C. Lin, “Home healthcare matching service system using iot: Third international conference, iotaas 2017, taichung, taiwan, september 20–22, 2017, proceedings,” in Jan. 2018, pp. 43–49, ISBN: 978-3-030-00409-5. DOI: 10.1007/978-3-030-00410-1_6.
- [25] “Wikipedia - microcontroller.” Accessed on November 26, 2023. (2019), [Online]. Available: <https://en.wikipedia.org/wiki/Microcontroller..>
- [26] “Raspberry pi.” Accessed on November 26, 2023. (), [Online]. Available: <https://www.raspberrypi.com/>.
- [27] “Esp32.” Accessed on November 26, 2023. (), [Online]. Available: [Amazon.co.uk](https://www.amazon.co.uk).
- [28] M. Sawh. “Ecg wearables: How they work and the best on the market.” Accessed on November 22, 2023. (2020), [Online]. Available: <https://www.wearable.com/health-and-wellbeing/ecg-heart-rate-monitor-watch-guide-6508>.
- [29] H. Kansal. “How ecg works on a smartwatch? how useful is it?— simplified.” Accessed on November 22, 2023. (2021), [Online]. Available: <https://wearablestouse.com/blog/2021/12/30/how-smartwatch-ecg-works/>.
- [30] “Pulse sensor amped - pimoroni.” Accessed on November 22, 2023. (), [Online]. Available: <https://shop.pimoroni.com/products/pulse-sensor-amped?variant=707189945>.
- [31] “Withnell sensors: Senseanywhere airosensor- wireless temperature monitoring.” Accessed on November 27, 2023. (), [Online]. Available: https://www.withnellsensors.co.uk/pages/products-and-services/data-loggers-thermocouples/senseanywhere-airosensor/?msclkid=edb5cf4f4920147e7110aa2bdb52f6d5&utm_source=bing&utm_medium=cpc&utm_campaign=SenseAnywhere&utm_term=temperature%20sensor&utm_content=Temperature%20sensor.

- [32] Z. Ali, *Introduction to ds18b20*, The Engineering Projects, Jan. 2019. [Online]. Available: <https://www.theengineeringprojects.com/2019/01/introduction-to-ds18b20.html>.
- [33] S. Jagdale. "Satellite iot: A new paradigm in iot connectivity." Accessed on November 25, 2023. (2022), [Online]. Available: <https://iot.eetimes.com/satellite-iot-a-new-paradigm-in-iot-connectivity/>.
- [34] *In Action: Planning and Leading Planning and Leading 2*, Accessed on November 26, 2023.
- [35] "Sdlc agile model." Accessed on November 26, 2023, W3 Schools. (), [Online]. Available: https://www.w3schools.in/sdlc/agile-model?utm_content=cmp-true.
- [36] E. Kate. "Comprehensive guide to the agile manifesto." Accessed on November 26, 2023. (2016), [Online]. Available: <https://www.smartsheet.com/comprehensive-guide-values-principles-agile-manifesto>.
- [37] B. Wald. "Top benefits of agile iot development." Accessed on November 26, 2023. (2020), [Online]. Available: <https://www.verytechnology.com/iot-insights/top-benefits-of-agile-iot-development>.
- [38] T. Paine. "Hree reasons to consider agile methodology for your iot effort - iot agenda." Accessed on November 26, 2023. (2016), [Online]. Available: <https://www.techtarget.com/iotagenda/blog/IoT-Agenda/Three-reasons-to-consider-agile-methodology-for-your-IoT-effort>.
- [39] D. Duka, "Adoption of agile methodology in software development," in *2013 36th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, 2013, pp. 426–430.

Appendices

Appendix A

Software Development Lifecycle (SDLC)

Agile Methodology is a popular approach in SDLC that emphasizes flexibility, collaboration and customer satisfaction [34] [35]. The development process is broken down into sprints (small, incremental builds or iterations). Such an iterative process allows developers to focus on the most important features at any given moment, rather than strictly following a pre-defined plan.

Agile Methodology is guided by its four core values [36]:

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer Collaboration** over contract negotiations
4. **Responding to change** over following a plan

The adoption of this methodology especially in IoT and software development has been widespread for several reasons.

- **IoT:** Agile methodologies support multiple protocols and networks utilised for building IoT solutions. It enables closer cooperation between IoT hardware and software development teams, fostering higher transparency [34]. This approach successfully addresses the complexity of connecting culture and technology in IoT implementation [38].
- **Software Development:** As per the 16th State of Agile Report by digital.ai, 78% of the respondents report that they have implemented or are planning to implement Agile at scale. Agile's emphasis on building working software quickly fostering cross-functional teams empowered to make decisions, and focusing on rapid iteration with continuous customer input are some of the reasons for its widespread adoption [39]. This approach not only follows better collaboration and incremental delivery but also provides early error detection and elimination of unnecessary work.

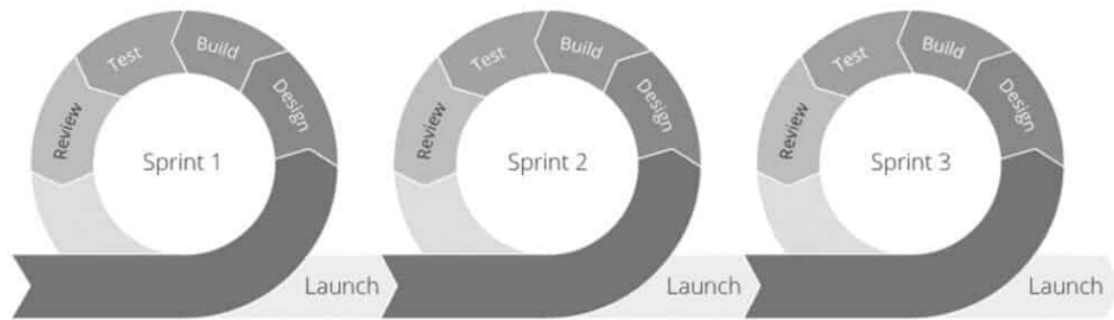


Figure A.1: Anatomy of Agile Approach [37]

Looking at the benefits of this approach, this project follows a similar approach.

Appendix B

Legal Considerations

B.1 Data Protection Regulations

The healthcare sector extensively employs information technologies, including wearable health monitoring devices and IoT (Internet of Things) devices, for real-time vital monitoring and data collection. These technologies involve the processing of significant amounts of personal data, necessitating adherence to data protection regulations.

Within the European Union (EU), personal data processing in healthcare is regulated by the General Data Protection Regulation (GDPR), which came into effect on May 25, 2018. The GDPR mandates obligations for data controllers and processors, ensuring the protection of individuals' personal data. Developers and providers of healthcare IoT devices must formulate privacy policies that align with GDPR requirements to ensure compliance.

B.2 Medical Regulations

Regulatory oversight in the UK for medical devices and IoT devices in healthcare is managed by the Medicines and Healthcare products Regulatory Agency (MHRA). Medical devices, including IoT devices used for vital monitoring, are categorized based on risk levels, ranging from low risk (Class I) to high risk (Class III). Even low-risk devices are subject to regulatory requirements, such as documentation preparation, clinical evaluations, and post-production review procedures. IoT devices intended for vital monitoring purposes fall under the scope of medical device regulations. Manufacturers must adhere to MHRA guidelines, which may include obtaining certifications and conducting clinical evaluations.

In compliance with regulatory guidelines, it is essential for developers and providers of IoT devices for vital monitoring to ensure transparency and accountability in data processing practices. Users should be informed about data collection processes and their rights regarding their personal data.

Appendix C

Ethical Considerations

In deploying IoT (Internet of Things) devices for passive monitoring in healthcare, several ethical considerations must be taken into account to ensure responsible and ethical use of technology.

- Ensuring data privacy and confidentiality is paramount, even in passive monitoring scenarios. Measures such as encryption, secure data transmission protocols, and access controls should be implemented to protect the confidentiality of collected data.
- While passive monitoring may not require active patient participation, it's essential to inform individuals about the presence and purpose of monitoring devices. Transparency about data collection activities helps build trust and respect for individuals' autonomy, even if explicit consent is not obtained.
- Clarifying data ownership and control is important, particularly regarding the rights and responsibilities of data stewards. Individuals should have clarity on who has access to their data and how it will be used, with mechanisms in place for individuals to access, modify, or delete their data as needed.
- In cases where algorithms are used to analyze collected data, transparency in algorithmic processes is crucial. Individuals should have visibility into how algorithms operate and make decisions to ensure accountability and mitigate risks of bias or discrimination.
- Consideration of the broader social implications of passive monitoring is essential. While these technologies offer potential benefits in terms of healthcare optimization, they may also raise concerns about surveillance and privacy infringement.

For the entire development and testing stage, it is advisable to utilize solely the developer's own health data rather than actual human health data to ensure ethical and responsible practices.

Appendix D

Adafruit IO Dashboard

Adafruit IO Dashboard was integrated with the system in order to test and validate the output from the sensors.

Integration with Adafruit IO is achieved through the MQTT protocol, where the device publishes sensor data to specific feeds. The corresponding dashboard on Adafruit IO is configured to subscribe to these feeds, thereby enabling real-time monitoring of health data.

To handle real-time communication and data transmission from Smart Patches to Adafruit IO Dashboard, MQTT (Message Queuing Telemetry Transport) protocol is used. MQTT is chosen for its lightweight nature and effectiveness in scenarios requiring minimal network bandwidth and device power consumption, which are critical in IoT contexts. Devices publish data to specific topics, which are dynamically subscribed to Adafruit IO Dashboard by the backend, enabling flexible and decoupled communication strategies.

To configure the Adafruit IO Dashboard, the following steps are followed:

1. **Create Feeds:** Feeds are created in Adafruit IO for each type of data that the Smart Patch will publish, such as heart rate and body temperature.
2. **Configure Widgets:** Widgets, such as line charts and gauges, are added to visualize data from specific feeds in the dashboard.
3. **Subscribe to Feeds:** Widgets are configured to subscribe to relevant feeds to ensure real-time data updates.
4. **Assign Alerts:** Alert thresholds are assigned for critical health parameters, providing instant notifications when readings exceed predefined limits.

In conclusion, the integration with Adafruit IO Dashboard provides a seamless way to test real-time health data from Smart Patch. With the lightweight and efficient MQTT protocol this system enabled accurate data transmission.

Appendix E

Privacy Policy

Privacy Policy for VitalMonitor

At VitalMonitor, one of our main priorities is the privacy of our users. This Privacy Policy document contains types of information that is collected and recorded by VitalMonitor and how we use it.

Information Collection And Use: We collect several different types of information for various purposes to provide and improve our Service to you. Personal data is collected for the sole purpose of providing the service and adhering to UK GDPR regulations.

Data Protection: The processing of personal data, such as the name, address, e-mail address, or telephone number, shall always be in line with UK GDPR. By using this Service, you agree to the collection and use of information in accordance with this policy.

Security: We value your trust in providing us your Personal Information, thus we are striving to use commercially acceptable means of protecting it.

Rights of Data Subjects: Data subjects have the right to confirmation, access, rectification, and erasure of personal data.

Changes to This Privacy Policy: We may update our Privacy Policy from time to time. Thus, you are advised to review this page periodically for any changes.

Contact Us: If you have any questions or suggestions about our Privacy Policy, do not hesitate to contact us at vvchoradial@sheffield.ac.uk.

Last updated: 08.04.2024

Figure E.1: Privacy Policy

Appendix F

Terms and Conditions

Terms & Conditions for VitalMonitor

Terms and conditions outline the rules and regulations for the use of VitalMonitor's services. By accessing and using our services, you accept and agree to be bound by the terms and provision of this agreement. If you do not agree to abide by the above, please do not use this service.

Service Description: VitalMonitor provides health monitoring services to organizations to track the health of their employees by monitoring beats per minute (BPM) and temperature. This data is stored securely in a database until the device assigned to a user is unassigned, the device is deleted, or the user is deleted.

User Privacy and Data Protection: Data protection is of a particularly high priority for the management of VitalMonitor. The use of our services is possible without any indication of personal data; however, processing of personal data could become necessary when an organization wishes to use our service. Our data processing protocols comply with the UK GDPR.

Confidentiality: VitalMonitor ensures that all employee health data is kept confidential and is only accessible to authorized personnel within the organization. We implement appropriate technical and organizational measures to safeguard the data from unauthorized access, alteration, or dissemination.

User Obligations: The organizations are responsible for ensuring that the use of VitalMonitor's services is in compliance with all applicable laws and for maintaining the confidentiality of health data within their organization.

Amendments to Terms: VitalMonitor reserves the right to modify these terms and conditions at any time.

Governing Law: These Terms shall be governed by the laws of the United Kingdom.

Last updated: 08-04-2024

Figure F.1: Terms and Conditions for VitalMonitor

Appendix G

Manual Testing

Table G.1: Manual Test Cases for VitalMonitor Web Dashboard

Test Case ID	Test Details	Expected Results	Pass/Fail
TC-WD01	Verify that admins can register an organization using valid details. - Navigate to registration form. - Input all required fields. - Submit form.	Admins are taken to home page.	Pass
TC-WD02	Verify admin login functionality with correct credentials. - Access login page. - Enter valid credentials. - Click login.	Admin is redirected to the dashboard home page.	Pass
TC-WD03	Verify the addition of a new device. - Log in as admin. - Navigate to "Add New Device" page. - Enter device details and submit.	Device addition confirmation message is displayed.	Pass
TC-WD04	Verify user-device assignment functionalities. - Log in and view user list. - Select a user without a device. - Assign a device and submit.	Device is listed as assigned to the user.	Pass
TC-WD05	Check the functionality of logging out. - Log in to the dashboard. - Click the logout button.	User is redirected to the login screen and session ends.	Pass

Table G.2: Manual Test Cases for Smart Patch Device

Test Case ID	Test Details	Expected Results	Pass/Fail
TC-SP01	Verify activation of the Smart Patch by pressing the button. - Ensure the device is off. - Press the activation button.	Device turns on (red light indicator) and starts transmitting data.	Pass
TC-SP02	Verify deactivation of Smart Patch by pressing the button again. - Ensure the device is on. - Press the button again.	Device turns off (red light off) and stops transmitting data.	Pass
TC-SP03	Verify continuous data transmission of heart rate and temperature. - Activate the device. - Monitor the data transmission for 10 minutes.	Continuous, accurate data transmission is observed.	Pass

Appendix H

Generation of design idea using Dall-E

DALL-E was instrumental in generating the design concept for a future Smart Patch. Multiple prompts were used to iteratively refine the final design output.

Prompts

1. Create an image of an IoT device that doesn't resemble a smartwatch.
2. Show a device that can be easily worn on the body and conveys a sense of health and wellness.
3. It should be compact, roughly the same size as a smartwatch, but distinctly different.
4. Make it resemble an earphone-like accessory, rather than a smartwatch.
5. It shouldn't have a microphone or speaker.
6. It should sit behind the ear.

The final output image, shown in Figure H.1, reflects these prompts.

Subsequently, the background was removed, and a plain blue box was placed behind the image, with the label “Generated using DALL-E.”



Figure H.1: Dall-E generated image

Appendix I

Abbreviations

AA - Double-A (battery size)
BLE - Bluetooth Low Energy
BMJ - British Medical Journal
CoAP - Constrained Application Protocol
GPIO - General Purpose Input/Output
HTTP - Hypertext Transfer Protocol
HCp - Healthcare Professionals
I/O or IO - Input/Output
IaaS - Internet of Things as a Service IC - Integrated Circuit
IDE - Integrated Development Environment
IDF - IoT Development Framework
ICU - Intensive Care Unit
IoT - Internet of Things
IoMT - Internet of Medical Things
IoTaaS - Internet of Things as a service IVD - In-Vitro Devices
LiIon - Lithium-Ion
LiPo - Lithium Polymer
LoRaWAN - Long Range Wide Area Network
MCU - Microcontroller Unit
MERN - MongoDB, ExpressJS, ReactJS, NodeJS
MQTT - Message Queuing Telemetry Transport
NoSQL - Not Only SQL (database)
OS - Operating System
PPG - Photoplethysmography
RAM - Random Access Memory
ROM - Read Only Memory
SDLC - Software Development Lifecycle
SoC - System on Chip TTN - The Things Network
USB - Universal Serial Bus